TRACE: Correlating Amcache, Prefetch and ShimCache with Constrained AI Reasoning for Early-Stage Incident Response

Capstone Final Report

Professor: Mark Shtern

Group Members: Adesina Laja, Mahdi Afshar, Abdulfatah Abdillahi, Hannan Karedia

Date: August 10, 2026

Table of Contents

Table of Tables **4**
Table of Figures **5**
Introduction **6**
Literature Review **9**
Methodology **16**
Design Components **19**
Stage 1 - Three Independent Parsers 19
Stage 2 - Cross-Artifact Correlation 19
Stage 3 - Threat Intelligence Corroboration 19
Stage 4 - AI Pass 1 (Metadata Triage) 20
Selection Gate 20
Stage 5 - AI Pass 2 (Evidence Reconciliation) 21
Stage 6 - Attribution Validation 21
Stage 6b - Deterministic Scoring Constraint 21
Stage 7 - Reporting 22
Implementation **23**
Tools and Technologies 23
Environment Setup 24
Hardware 24
Software prerequisites 24
Python dependencies 25
Installation and launch 25
Configuration Details 26
Pipeline Implementation 27
Stage 1 - Artifact Parsing 27
Stage 2 - Cross-Artifact Correlation 28
Stage 3 - Threat Intelligence Corroboration 30
Stages 4 and 5 - The Reasoning Engine 30
Prompt Construction and Constraint 31
Stage 6 - Attribution Validation 32
Stage 6b - Deterministic Scoring 33
Stage 7 - Reporting and Delivery 33
Security Features 34

Implementation Challenges 35
A capacity ceiling on prompt length 35
Configuration silently overridden by the environment 35
Absent data misread as negative data 36
Network failures cached as verdicts 36
A false forensic signal, identified by direct verification 37
Functional Prototype Demonstration 37
Experimental Assessment 39
Experiment Test 2 – Benign-Only False Positive Stress Test — Model Comparison (Qwen vs. Gemini) 49
Experiment Test 3 – Gemini False-Positive Rate at Maximum Thinking Depth (Benign-Only Dataset) 56
Results & Discussion 57
Summarize Results 57
Performance Analysis 58
Security Analysis 59
Compare with Existing Solutions 60
Discussion and Interpretation 61
Limitations 62
Technical Limitations 62
Data Limitations 63
Environmental Constraints 64
Threats to Validity 64
Conclusion and Future Work 65
Research Summary 65
Key Findings 65
Contributions 67
Future Improvements 67
References 68
Appendix A 72
Tools Used 72
Malware Execution / Evidence Gathering 73

Table of Tables

Table 1 - TRACE Components and Implementation Choices 24
Table 2 - Software Prerequisites and Versions 25
Table 3 - Python Dependencies and Their Roles 25
Table 4 - Reasoning engine parameters 26
Table 5 - Threat intelligence parameters 27
Table 6 - Scoring parameters 27
Table 7 - Models Under Test 40
Table 8 - Datasets Under Test 41
Table 9 - Comparison for Malware Datasets 46
Table 10 - Comparison table for Benign dataset results 52
Table 11 - Metrics for benign dataset results 52
Table 12 - Findings from both models ran against the benign dataset 55
Table 13 - Tools Utilized to gain initial acquisition 72
Table 14 - Artifact Locations Referenced 73

Table of Figures

Figure 1 - TRACE Analysis Process Flow 17
Figure 2 - Isolated Bridged Network Topology 39
Figure 3 - Gemini D1 Dataset result 42
Figure 4 - Gemini D2 Dataset result 43
Figure 5 - Qwen D1 Dataset result 44
Figure 6 - Qwen D2 Dataset result 45
Figure 7 - Gemini Benign Dataset result 50
Figure 8 - Qwen Benign Dataset result 51
Figure 9 - Qwen & Gemini false positive rate graph 53
Figure 10 - Gemini ran against benign dataset with high thinking level 57
Figure 11 - Malware Executed on the Isolated Victim Machine 74
Figure 12 - Creating a Physical Drive Image in FTK Imager 75
Figure 13 - FTK Imager Destination Folder and Image File Name 75
Figure 14 - FTK Imager Imaging in Progress 76
Figure 15 - Drive/Image Verify Results 77
Figure 16 - Image Segments in Windows Explorer 78
Figure 17 - Loading the Evidence Image as a Source in FTK Imager 79
Figure 18 - Extracting Windows Artifacts from the Evidence Image 80

Introduction

When a Windows machine is suspected of compromise, the artifacts that record what ran on it are recovered quickly. Amcache, Prefetch and ShimCache are collected in minutes with mature, well-understood tools, and a single workstation typically yields several thousand entries across the three. What follows takes far longer. An analyst must read those entries, recognise which executables are ordinary system activity and which are not, check suspicious hashes against external sources, and reconstruct which programs actually executed rather than merely existed on disk. The bottleneck in early-stage incident response is rarely the acquisition of evidence; it is the interval between having the evidence and forming a judgement about it.

That interval is an obvious candidate for automation, and it is increasingly common to see language models proposed for it. The difficulty is that a language model is a poor fit for forensic work in its default form. It is probabilistic where forensic conclusions must be reproducible, it will state a fabricated MITRE ATT&CK technique identifier as confidently as a real one, and it tends to treat unfamiliar file names as suspicious and familiar ones as safe — which is precisely the assumption adversaries exploit when they name malicious binaries after legitimate system components. A tool that hands such a model unconstrained authority over findings produces output that looks authoritative and cannot be defended.

The primary objective of this project was to develop TRACE (Threat Reconnaissance and Artifact Correlation Engine) for educational purposes: a working, fully documented forensic triage tool whose internal reasoning is open to inspection at every stage. TRACE is not offered as a production incident-response product, nor as a replacement for established commercial or open-source DFIR tooling. It is a study of a specific question — whether a small language model running on ordinary hardware can contribute usefully to forensic triage when it is deliberately bounded on both sides by deterministic logic — built so that the answer, and the reasoning behind every design decision that produced it, can be examined by someone learning the subject.

The motivation is that tools of this kind are almost always opaque to the people who most need to understand them. A student or early-career practitioner using a commercial triage platform sees a severity score and a verdict. They do not see which artifact contributed which

piece of evidence, how competing signals were weighed, what happens when an external service is unreachable, or why one file scored 9.0 and another 6.5. The reasoning that matters educationally is exactly the reasoning that is hidden.

TRACE was built to invert that. Every stage of its pipeline is a separate, single-purpose module that can be read independently. Every finding includes the model's written justification alongside a plain-language statement indicating whether its score was the model's own judgement or raised by a deterministic rule. Every ATT&CK attribution is validated against the authoritative catalogue, and the number of rejected identifiers is reported as a property of the run. The design intent is that a reader can follow a single file from a row in a CSV export through correlation, external corroboration, two reasoning passes and two validation layers, and understand at each point what was known about it and what was decided.

A second motivation is that the failures encountered while building such a tool are themselves instructive and are normally discarded. Several of this project's more useful outcomes were negative results — a prompt revision that improved precision while measurably destroying recall, a forensic indicator that turned out not to be an indicator at all, a caching bug that silently marked malicious files clean. These are reported in full rather than quietly fixed, because understanding why a plausible approach failed is more educationally valuable than a clean description of the approach that eventually worked.

A complete, functioning prototype was produced. TRACE ingests CSV exports of the three artifact types, normalizes them to a single representation with all timestamps in UTC, correlates observations of the same executable across artifacts so each program is assessed exactly once, corroborates every unique file hash against three independent threat-intelligence services while transmitting nothing but the hash, and assesses the result through two separate reasoning passes executed on a locally hosted model. Between those passes sits a deterministic Selection Gate that admits an entry for full review if *either* the model flagged it *or* external intelligence returned a signal — a disjunction that prevents either evidence source from independently suppressing a finding. The model's output is then constrained by independent validation of its technique attributions and by a scoring rule that can raise a severity but never

lower one. Findings are delivered as a PDF forensic report, a CSV export and a dashboard view, all generated offline.

Across the four evaluations conducted for this project — two independent real-malware datasets (D1 and D2, each combining MalwareBazaar samples with a locally crafted Meterpreter payload), a benign-only false positive stress test, and a fabricated scenario modeled on a documented ransomware attack chain the prototype consistently identified the vast majority of known-malicious samples, including payloads that no antivirus engine had catalogued and were therefore detectable only from artifact evidence rather than hash reputation. The same evaluations also produced a measurable number of false positives, most concentrated in the benign-only stress test, where the reasoning engine occasionally misjudged legitimate installer and setup executables despite clean corroboration from every threat-intelligence source. Investigating these false positives yielded the project's more interesting technical findings: that instruction length imposes a real capacity ceiling on small local models, that the PE compile-timestamp field is not a reliable tampering indicator on modern Windows binaries because deterministic build toolchains write a content hash into it, and that the distinction between a data field being absent and being empty must be represented explicitly or a reasoning engine will misinterpret it. Substituting the reasoning engine across two architecturally different models a small model running locally and a larger model running as a hosted API further showed that neither model was categorically superior: each converged on a different point along the same precision-recall tradeoff, while the deterministic scaffolding surrounding the model (correlation, threat-intelligence corroboration, the Selection Gate, and attribution validation) remained the more consistent contributor to overall accuracy across every run.

The architecture that produced these results is described in the Methodology section, which outlines the layered design, the pipeline stages, and the rationale for separating responsibilities between them. The Implementation section then maps that design onto the specific environment, libraries, configuration and prompt construction used to build it, and documents the problems encountered along the way.

Literature Review

Building TRACE required studying four distinct bodies of work: the forensic artifacts that supply its evidence and the tools that parse them, the existing triage platforms it would sit alongside, the detection techniques and intelligence sources it would draw on, and the language-model technologies that make its reasoning stage possible at all. This section describes what was studied in each area, what was adopted, and — often more instructive — what was examined and deliberately rejected. The organising principle throughout is the connection between a body of prior work and a decision visible in the finished tool.

The starting point was the evidence itself. Windows records program execution across several artifacts, none of which is individually sufficient. Amcache, a registry hive, records that Windows inventoried a file and preserves a SHA-1 hash together with signing metadata, but does not establish that the file ran. Prefetch records that a program executed, with run counts and up to eight execution timestamps, yet carries neither hash nor publisher information. ShimCache, the application compatibility cache, records that Windows evaluated a binary, which may or may not indicate execution depending on the Windows version. The governing process guidance for incorporating such artifacts into an investigation is NIST Special Publication 800-86 [1], which sets out collection, examination, analysis and reporting as distinct phases and insists that conclusions remain traceable to the data producing them. NIST SP 800-61 [2] situates that work within the wider incident-handling lifecycle and identifies detection and analysis as the phase consuming the greatest share of analyst time, while ISO/IEC 27037 [3] addresses chain of custody for digital evidence. Studying these three together produced two decisions carried through the whole design: that every finding must record a written justification and the provenance of its score, and that case metadata must travel with the findings into the report. The complementary nature of the artifacts themselves produced a third — that correlation across artifacts, rather than analysis of any one, would be the tool's central evidential mechanism.

Parsing those artifacts is a solved problem, and studying the available parsers led directly to a decision not to reimplement them. Eric Zimmerman's forensic tool suite [4] provides mature, widely trusted parsers for each artifact type — AmcacheParser, PECmd and AppCompatCacheParser — each emitting CSV. Their standing in the practitioner community is

such that reimplementing their parsing logic would have introduced risk without benefit, and would have made TRACE's output harder rather than easier to trust. TRACE therefore consumes their CSV output rather than raw hives, which keeps the conversion from binary artifact to tabular data in tooling that an examiner already trusts and can independently verify. The cost of that decision is a manual pre-processing step; the benefit is that TRACE's input is a file the analyst can open and inspect before submitting it.

The natural next question was how existing tools combine artifacts once parsed, and the dominant answer is timeline construction. Plaso, the successor framework to Guðjónsson's log2timeline [5], parses many artifact types into a single chronologically ordered stream — the "super timeline" — allowing events from disparate sources to be read together. The insight that artifacts become far more informative once normalised into a common representation is one TRACE adopts directly, and is why every parser converts timestamps to UTC before emitting a record. The approach has a well-understood cost, however: a super timeline across a single workstation routinely produces hundreds of thousands of entries, which returns the analyst to the original problem of deciding which entries matter. Timeline analysis solves normalisation and ordering but not prioritisation. Studying this led to the most consequential structural decision in TRACE's design: rather than correlating by time, it correlates by entity, merging every observation of the same executable into one record carrying the union of what all three artifacts observed. Time is retained within that record as execution evidence rather than serving as the organising axis. The reasoning is that an analyst triaging a host needs to know which programs warrant attention, not what happened in which order, and entity-centric grouping yields one decision per program rather than one per event.

Several established platforms address collection and triage more broadly, and each was examined for what it does well and what it leaves to the analyst. Autopsy, the graphical front end to Brian Carrier's Sleuth Kit [6], provides comprehensive disk analysis with an ingest-module architecture that inspired TRACE's own separation of pipeline stages into independently replaceable components. KAPE [7] focuses on targeted collection and processing, applying parsers to collected artifacts at speed, and demonstrates the value of a narrow, well-defined artifact scope — a principle TRACE follows by supporting three artifact types thoroughly rather than many superficially. Velociraptor [8] provides endpoint visibility and artifact collection at

fleet scale through its own query language. Commercial triage products such as Magnet AXIOM and Cyber Triage automate much of the analysis and present ranked findings. What these tools share, and what motivated this project, is that the reasoning behind a prioritisation is generally not exposed: an analyst sees that a file was flagged, but not which evidence contributed, how competing signals were weighed, or what the tool would have concluded had one source been unavailable. For a tool built primarily for educational purposes, making that reasoning inspectable was the point rather than an afterthought.

The detection techniques these platforms rely on were studied next. Signature matching — hash comparison against known-bad sets, and pattern matching through YARA rules [9] — is precise, fast and entirely deterministic, and remains the correct approach where a signature exists. Detection-as-code approaches such as Sigma [10] extend the same idea to log and event data with portable, vendor-neutral rules. A survey of digital triage methods and tools [11] confirms that most automated triage reduces in practice to keyword search, hash matching or rule evaluation. The shared limitation is structural rather than incidental: none of these techniques can reason about evidence for which no rule has been written. This is the specific weakness TRACE's first reasoning pass targets, and the effect is observable — in evaluation, TRACE surfaced a masquerading executable carrying zero antivirus detections, identified solely from its location and naming, which no signature-based approach could have flagged.

The alternative to signatures in the research literature is supervised machine learning, and studying that literature produced a decision not to pursue it. Pendlebury et al.'s TESSERACT [12] demonstrated that reported performance in malware classification research is systematically inflated by spatial bias, where the ratio of malicious to benign samples in evaluation does not reflect deployment, and temporal bias, where training and testing splits let a classifier learn from data that would not have existed at prediction time. Their re-evaluation of three published Android classifiers across 129,000 applications confirmed that headline F1 scores approaching 0.99 do not survive realistic constraints. Two consequences followed for this project. A classifier trained on labelled artifact data would require continual retraining against concept drift and honest temporal evaluation, which was beyond the scope of a capstone; and the spatial-bias argument set the expectation for TRACE's own results, since a real host contains overwhelmingly benign software and even a low false-positive rate therefore produces many

false positives in absolute terms. That expectation proved accurate, and it is why the evaluation reports precision and recall separately rather than a single accuracy figure.

External corroboration was studied both as a set of available services and as a body of research on their reliability. VirusTotal [13], OPSWAT MetaDefender and AlienVault OTX each expose REST APIs permitting lookup by file hash, returning per-engine detection counts, family attribution and community context. The academic literature on these sources is considerably more cautious than practitioner usage suggests. Sebastián et al. built AVClass [14] precisely because vendors use incompatible naming vocabularies, and their evaluation over eight million samples could label only 81 per cent, since nearly one sample in five had fewer than two engines agreeing on a non-generic family name. Zhu et al. [15] showed in a large-scale study that engine verdicts are not stable over time, with samples receiving materially different verdicts weeks apart as signatures propagate, and cautioned against treating any single scan as ground truth. These findings shaped the corroboration layer in three concrete ways: three independent services are queried rather than one, because cross-source agreement is the meaningful unit; the confirmed-malware threshold is expressed as a count of independent engines rather than any single vendor's verdict, following AVClass's plurality logic; and multi-engine consensus can raise a severity but never lower one, because a clean result may mean only that a sample is not yet catalogued. The same literature justified querying every hash unconditionally rather than only those the reasoning engine flagged, since gating external lookups behind a model's opinion compounds one source of error with another.

Expressing conclusions about adversary behaviour required a shared vocabulary, and MITRE ATT&CK supplies it. The framework's design rationale is set out by Strom et al. [16], who describe a knowledge base grounded in real-world observation rather than theoretical attack modelling, and emphasise that its value derives from being empirically sourced and consistently structured. Two properties proved decisive. ATT&CK is a closed, enumerable set of identifiers, which means a model can be given a menu to choose from rather than being asked to recall an identifier unaided. And the catalogue is distributed in STIX [17], a structured, machine-readable format, which means every returned identifier can be independently verified against the authoritative source. TRACE implements both as separate defences: a runtime-generated menu

of forty execution-observable techniques constrains what the model is asked to produce, while post-hoc validation against the full parsed catalogue checks what it actually produced.

The reasoning stage rested on a set of technologies studied specifically for this project. Running a capable language model on an 8 GB consumer GPU depends on quantisation — reducing the numeric precision of model weights so the model fits in available memory. The GGUF format and the llama.cpp inference engine [18] made this practical, and LM Studio [19] provides a graphical interface over the same capability while exposing an OpenAI-compatible HTTP API. That last property mattered more than it first appears: because LM Studio speaks the same protocol as hosted providers, TRACE could be written against the standard OpenAI Python client and later addressed at a hosted model by changing a single dispatch function, which is what made a comparison run possible without restructuring the pipeline. The Q4_K_M quantisation level was selected because it resides wholly within 8 GB of video memory, avoiding the partial CPU offload that increases per-call latency by roughly an order of magnitude. Michelet and Breitinger [20] examined whether locally hosted models can assist in producing digital forensic reports and found usable output while keeping case data on the examiner's own hardware, which supported the choice of local inference on evidential-confidentiality grounds; their observation that smaller local models produce noticeably weaker output than hosted frontier models anticipated a constraint encountered directly during development.

Prompt engineering was studied as a technique in its own right, since the reasoning stage's behaviour is determined almost entirely by its instructions. Chain-of-thought prompting [21] demonstrates that decomposing a task into explicit intermediate steps improves reasoning reliability — a finding TRACE applies structurally rather than textually, by splitting assessment into two passes with different evidence available to each rather than instructing the model to reason step by step inside a single call. Retrieval-augmented generation [22] constrains a model by supplying authoritative content at inference time instead of relying on parametric memory, and TRACE applies the same principle in restricted form by placing the ATT&CK menu and the curated antivirus summary directly in the prompt, precisely so the model is never asked to recall facts it might fabricate. The separation of system and user prompts, and the specification of an exact JSON output schema, follow conventional practice for structured generation; what proved necessary in addition was defensive parsing, since small local models routinely emit reasoning

blocks, Markdown code fences and explanatory prose around an answer that was specified to be a bare array.

The limitations of these models were studied alongside their capabilities. Ji et al. [23] survey hallucination across generation tasks, distinguishing intrinsic hallucination, which contradicts the source, from extrinsic hallucination, which the source cannot verify, and make the observation most relevant to security applications: hallucinated content is indistinguishable from correct content by fluency or confidence, so a fabricated ATT&CK identifier is expressed exactly as assuredly as a real one. Scanlon et al. [24] evaluated GPT-4 across digital forensic use cases including artefact understanding, evidence searching, anomaly detection and incident response, concluding that the technology is a capable assistant but unreliable as an autonomous determiner of findings, and documenting confident factual errors on forensic specifics. Their position — that the analyst should retain determination authority — is one TRACE adopts architecturally rather than merely procedurally.

The security operations literature supplied the sharpest statement of the failure mode to design against. Habibzadeh et al. [25] survey language-model applications across alert triage, threat intelligence summarisation and incident response, identifying alert fatigue as the driving motivation. More directly useful was an empirical evaluation of general-purpose models for alert classification and prioritisation [26], which tested eight models against manually labelled alerts and found that prioritisation proved substantially harder than classification: several models achieved strong recall on high-severity alerts, yet precision remained consistently low across every model evaluated. Language models applied to security triage, in short, over-flag. Knowing this in advance is what motivated the two-pass structure, in which the second pass reconciles the first against external evidence before any verdict is recorded. A longitudinal study of human–AI collaboration in operational security operations centres [27] found that analysts given free access to model assistance used it for sensemaking and context-building rather than for high-stakes determinations — practitioners converging by instinct on the division of labour TRACE enforces in software. The OWASP Top 10 for Large Language Model Applications [28] provided the security-engineering framing, identifying overreliance on model output and inadequate output handling among the principal risks of model-integrated systems; TRACE's validation and scoring layers are direct mitigations of both.

The supporting implementation technologies were selected on more prosaic grounds but are worth recording. Pandas [29] handles CSV ingestion in all three parsers, chosen for its tolerance of malformed input and its explicit control over type inference, both of which proved necessary — forcing string types prevents SHA-1 values being corrupted into floating-point representations, and disabling default NA conversion keeps a literal "NA" distinguishable from a genuinely absent field. FastAPI [30] provides the service layer, chosen for its automatic request validation and generated API documentation, and ReportLab [31] generates the PDF report with no external fonts, stylesheets or network calls, which allows the deliverable to be produced on an isolated workstation.

Taken together, this study produced a consistent picture. Deterministic artifact analysis is reliable and defensible but cannot reason about evidence for which no rule has been written. Statistical classification generalises beyond written rules but demands labelled data, degrades under drift, and is routinely evaluated under conditions that overstate its performance. Language-model automation reasons flexibly about unfamiliar evidence and produces human-readable justification, but hallucinates confidently and characteristically over-flags. Existing triage platforms are capable but generally do not expose the reasoning behind a prioritisation. TRACE's design responds to each of these: it consumes trusted parsers rather than reimplementing them, correlates by entity rather than by time, queries multiple intelligence sources and thresholds on their agreement, reasons about evidence through a language model constrained by a closed attribution menu and independent validation, and subjects the model's score to a deterministic rule it cannot override. The disjunctive Selection Gate expresses the underlying principle most directly: by admitting an entry for full review if either the model flagged it or external intelligence returned a signal, neither the probabilistic component nor the external source can independently suppress a finding.

A final observation concerns what this body of work does not contain. The literature and documentation studied here is overwhelmingly concerned with whether an approach works. Very little of it examines the failure modes encountered in building one, and negative results are seldom reported in detail. This project treats several such results as findings in their own right — a prompt revision that improved precision while measurably destroying recall, a forensic indicator that proved on direct verification not to be an indicator at all, and a caching defect that

silently inverted verdicts — on the grounds that for a tool built to be understood, an account of why a plausible approach failed carries more instructional value than a clean description of the approach that eventually worked.

Methodology

This research proposes a forensic triage architecture that addresses the operational bottleneck between artifact collection and analyst judgement in Windows incident response. To triage program-execution artifacts without exposing evidence to third-party services, and without allowing an unverified language model to determine findings unilaterally, the architecture is strictly segmented into four layers, each with a single responsibility and no authority over the others.

The ingestion and normalisation layer parses each artifact type independently and reduces three incompatible record formats to one uniform representation with all timestamps expressed in UTC. The correlation layer merges observations of the same executable across artifacts into a single record, so that each program is assessed once and no judgement is made from partial metadata. The corroboration layer tests every file hash against multiple independent security vendors, transmitting only the hash so that no file content, path or host identifier crosses the network. The reasoning and constraint layer performs assessment in two separate passes and then subjects the result to independent validation of its technique attributions and a deterministic floor on the severity of externally confirmed malware.

Evidence never leaves the analyst workstation, and no single component is permitted to produce a finding on its own. Every reported finding is the product of at least two independent mechanisms agreeing - or, where they disagree, of a deterministic rule adjudicating between them on the record.

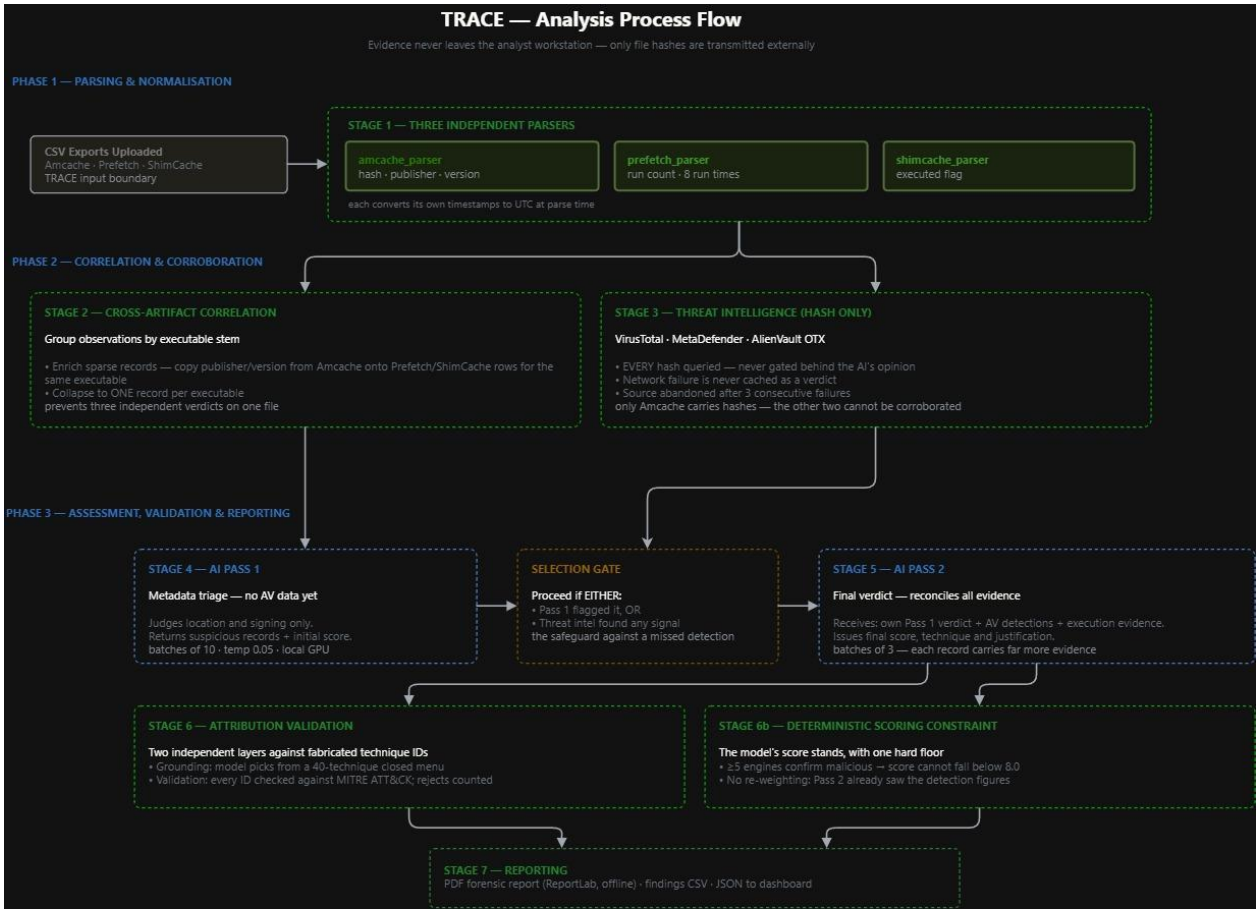


Figure 1 - TRACE Analysis Process Flow

Figure 1 traces a single analysis from the moment artifacts are submitted to the moment a report is produced. The pipeline is divided into three phases, and each phase hands a progressively richer representation of the same evidence to the next.

In Phase 1, TRACE accepts only CSV exports produced by established forensic parsers. Three independent parsers - one per artifact type - read their own file, extract the fields TRACE reasons about, and convert every timestamp to UTC before passing anything downstream. Normalization happens inside each parser rather than in a shared module, because each artifact records time differently and the conversion is only safe when it is written with knowledge of that specific format. The output of this phase is a single uniform stream of records in which the origin artifact is no longer structurally significant.

Phase 2 runs two operations that are deliberately independent of each other and of the reasoning engine. Cross-artifact correlation groups every observation by executable and merges them into one record, so that a program recorded in all three artifacts is judged once rather than three times, and so that identity metadata held by only one artifact enriches the sparse records from the others. In parallel, threat intelligence corroboration queries three external services for every unique file hash. This lookup is unconditional: it is never gated behind the reasoning engine's opinion, because a file the model considers unremarkable may still be a known-malicious sample. Only the hash is transmitted; no file content, path or host identifier leaves the workstation.

Phase 3 performs assessment in two separate reasoning passes with a deterministic decision between them. Initial experiments of relying on a singular reasoning pass resulted in an increased rate of false positives. To better mitigate these outcomes and ensure the model could make an informed judgement, we split the assessment into two separate reasoning passes (*Pass 1* and *Pass 2,* respectively**)**. The first pass sees metadata alone and produces an initial triage judgement. The Selection Gate then decides which entries deserve a full review, admitting an entry if either the first pass flagged it or threat intelligence returned a signal - the disjunction is what prevents a confirmed-malicious file from being discarded because the model found its metadata ordinary. The second pass receives its own earlier verdict together with the antivirus evidence and the correlation evidence, and issues a final judgement having seen everything. Two validation layers then constrain that output before it reaches the analyst: technique attributions are checked against the authoritative ATT&CK catalogue, and a deterministic scoring rule enforces a minimum severity for files that external engines independently confirm as malicious. The reporting stage renders the surviving findings, risk-sorted, into a PDF report, a findings CSV and JSON for the dashboard.

The communication pattern across the diagram is deliberately one-directional. No stage asks a later stage for information, and no stage revises an earlier stage's output in place; each stage consumes what precedes it and emits a new, more complete representation. The single probabilistic component - the reasoning engine - is bounded on both sides by deterministic logic: the evidence it receives is assembled deterministically, and the score it returns is constrained deterministically.

Design Components

Each stage in Figure 1 is implemented as a self-contained component with a single responsibility, so that any individual stage can be replaced, tested or reasoned about without altering the rest of the pipeline.

Stage 1 - Three Independent Parsers

What it is: The ingestion layer. Three separate parsers, one for each supported artifact type: application inventory, execution traces, and compatibility cache entries.

How it works: Each parser reads the CSV export for its own artifact and resolves column names case-insensitively against a list of known aliases, so that CSVs produced by different tool versions are handled without configuration. It extracts only the fields the pipeline reasons about, records whether a field was genuinely absent from the source rather than merely empty for that row, and converts every timestamp it emits into UTC. Rows carrying no usable identifier are discarded rather than propagated. Because each parser owns its own normalisation, adding support for a further artifact type requires writing one new file and changes nothing that already exists.

Stage 2 - Cross-Artifact Correlation

What it is: The component that converts three separate views of a machine into one view of each program on it.

How it works: Every record is reduced to a matching key derived from the executable name, because the three artifacts express file paths in incompatible notations and the name is the only reliable common identifier. Records sharing a key are merged into a single enriched record: identity metadata held by only one artifact is copied onto the others, execution evidence is combined, and every distinct path observed for that name is retained. This achieves two things. It ensures the reasoning engine is never asked to judge a program from a bare filename when richer metadata for it exists elsewhere in the dataset, and it guarantees each program is assessed exactly once, eliminating the possibility of three independent verdicts on the same file.

Stage 3 - Threat Intelligence Corroboration

What it is: The external corroboration layer, which tests the pipeline's own observations against the judgement of independent security vendors.

How it works: Every unique file hash in the dataset is submitted to three independent services. Only the hash is transmitted, which preserves evidential integrity: the services learn nothing about the file's name, location or host. Results are cached so repeated analyses do not re-query the same hash. The component distinguishes carefully between a service returning "not found" and a service being unreachable — the former is a legitimate result and is cached, the latter is a failure and is never cached, because storing a network error as a verdict would silently mark a malicious file as clean on every subsequent run. If a service fails repeatedly it is abandoned for the remainder of the analysis, so an unreachable provider degrades coverage rather than stalling the investigation.

Stage 4 - AI Pass 1 (Metadata Triage)

What it is: The first reasoning pass, which forms an independent opinion from file metadata alone.

How it works: Records are submitted to a locally hosted language model in batches, with a shared rulebook instructing it to judge location, signing status and naming rather than familiarity with a product name. It is given no antivirus evidence at this stage. This is a deliberate design choice: withholding external verdicts forces the pass to reason from the artifact evidence itself, which is what allows TRACE to surface files that no antivirus vendor has yet catalogued. Sampling temperature is set near zero for repeatability, and the pass returns only the entries it considers suspicious, each with an initial score and a proposed technique attribution.

Selection Gate

What it is: A deterministic filter that decides which entries proceed to full review.

How it works: An entry advances if the first pass flagged it or if threat intelligence returned any signal for its hash. The disjunction is the important property. Were the gate to require the model's approval, a file the model judged unremarkable on metadata alone would never be reconciled against the antivirus evidence that already identifies it as malicious. Were it to require an intelligence signal, novel malware with no external record would be discarded. Admitting on either condition means neither evidence source can independently suppress a finding, and the count of entries admitted on intelligence alone is recorded as a measure of what the reasoning pass would otherwise have missed.

Stage 5 - AI Pass 2 (Evidence Reconciliation)

What it is: The final reasoning pass, which issues the verdict recorded in the report.

How it works: Each candidate is resubmitted to the same model in smaller batches, because each record now carries substantially more context. The pass receives its own first-pass verdict, the curated antivirus evidence for the file's hash, and the cross-artifact execution evidence establishing whether the program actually ran and how often. It is instructed on how to weigh these against one another — that consensus across many independent engines is strong evidence in either direction, that an absent record means untested rather than clean, and that execution corroborates suspicion without being suspicious in itself. It then issues a final score, technique and written justification. Presenting the evidence to the model rather than applying it as an arithmetic adjustment afterwards means the recorded reasoning genuinely reflects the evidence the verdict was based on.

Stage 6 - Attribution Validation

What it is: A two-layer defence against fabricated technique attributions.

How it works: The first layer is preventative: both passes are given a closed menu of execution-observable techniques generated at runtime from the authoritative ATT&CK data and instructed to choose from it or return nothing, so the model is never asked to recall a technique identifier unaided. The second layer is independent verification: every identifier returned is checked against the full catalogue regardless of the menu, sub-technique identifiers that cannot be resolved fall back to their parent, and anything unrecognised is rejected and counted. The count is retained in the analysis metadata, making the reasoning engine's attribution reliability an observable property of each run rather than an assumption.

Stage 6b - Deterministic Scoring Constraint

What it is: The constraint layer separating the model's judgement from the final recorded severity.

How it works: The score returned by the second pass is accepted as issued, subject to one rule: where a defined number of independent antivirus engines confirm the hash as malicious, the score cannot fall below a fixed floor. The constraint operates in one direction only — it can raise a score, never lower one — and functions as a safeguard against the model under-reacting to unambiguous external evidence. No weighted blending of the model's score with detection ratios

is performed, because the second pass has already seen those figures and combining them again would count the same evidence twice. Every finding carries a plain-language statement recording whether its score was the model's own or was raised by this rule, so the provenance of each severity is visible to the analyst.

Stage 7 - Reporting

What it is: The delivery layer, producing the analyst-facing outputs.

How it works: Surviving findings are sorted by risk and rendered into three artefacts: a structured PDF report containing the executive summary, the risk-sorted findings with their reasoning, the execution evidence established by correlation, the indicators of compromise, the chain-of-custody record and a statement of methodology; a findings CSV for ingestion into other tooling; and JSON for the dashboard. Report generation is performed entirely offline, so the deliverable can be produced on an isolated workstation.

Implementation

The methodology described the architecture in terms of what each layer is responsible for and why the responsibilities are divided as they are. This section describes how that architecture was built: the environment it runs in, the libraries chosen for each stage, the configuration that governs its behaviour, the security properties the implementation preserves, and the problems encountered in making it work. The pipeline is implemented as ten Python modules under backend/pipeline/, each corresponding to one stage of Figure 1, coordinated by a single orchestrator and exposed through a FastAPI service.

Tools and Technologies

Component	**Implementation Choice**	**Purpose & Context**
Artifact Acquisition	Eric Zimmerman's forensic parsers (***AmcacheParser***, ***PECmd***, ***AppCompatCacheParser***)	Convert raw registry hives and prefetch files into tabular CSV exports. Executed as an operator-run pre-processing step, external to the analysis engine, so parsing remains independently auditable.
Artifact Parsing Layer	Python 3.11 with ***pandas***, ***python-dateutil***	Three independent parsers mapping each artifact's native schema onto the shared observation model with UTC normalization.
Cross-Artifact Correlation	Custom Python correlation module.	Indexes observations by executable stem, enriches sparse records, and collapses each executable to a single assessment record.
Threat Intelligence Corroboration	VirusTotal, MetaDefender Cloud, AlienVault OTX (REST APIs over HTTPS)	Hash-only lookups returning per-engine detection counts, malware family attribution and community threat context.
Constrained Reasoning Engine	LM Studio serving a quantised Qwen instruct model (Q4_K_M) via an OpenAI-compatible local API, GPU-offloaded to an NVIDIA RTX 3070 (8 GB)	Performs both assessment passes entirely on the analyst workstation. Quantisation selected so the model resides wholly in GPU memory, avoiding CPU offload latency.

Attribution Catalogue	MITRE ATT&CK Enterprise STIX bundle (local copy)	Provides the curated technique menu and the authoritative set against which every returned attribution is validated.
Orchestration & Interface	FastAPI with Uvicorn; static HTML/JavaScript dashboard	Exposes artifact upload, executes the pipeline, and serves findings, the PDF report and the CSV export
Report Generation	ReportLab	Produces the analyst-facing PDF entirely offline, with no external rendering dependency.

Table 1 - TRACE Components and Implementation Choices

Environment Setup

TRACE runs entirely on a single analyst workstation. No component requires a server, a container runtime or an internet-facing deployment, which is a consequence of the design rather than a convenience: an incident-response tool that processes evidence must be able to run on an isolated machine.

Hardware

The development and evaluation workstation used an NVIDIA RTX 3070 with 8 GB of VRAM. This constraint drove the single most consequential implementation decision, the choice of model quantisation. A quantised Q4_K_M build of a Qwen instruct model was selected because it resides wholly within 8 GB of video memory. A larger quantisation would exceed available VRAM and force partial CPU offload, which increases per-call latency by roughly an order of magnitude — unacceptable for a pipeline that issues one model call per batch across two passes. The hardware therefore bounds the model capacity available to the reasoning stage, which in turn bounds how long an instruction set that model can reliably follow.

Software prerequisites

Component	**Version/Detail**
Python	3.11
LM Studio	Serving a Q4_K_M Qwen instruct model with

	GPU offload enabled
Eric Zimmerman's tools	AmcacheParser, PECmd, AppCompatCacheParser — operator-run pre-step.
MITRE ATT&CK Enterprise STIX bundle	Local copy at *data/mitre/enterprise-attack.json*

Table 2 - Software Prerequisites and Versions

Python dependencies

Library	**Used by**
pandas	CSV ingestion in all three parsers
python-dateutil	Timestamp parsing and UTC normalisation
requests	Threat-intelligence HTTPS clients
openai	Client for LM Studio's OpenAI-compatible local API
python-dotenv	Configuration loading from .env
reportlab	Offline PDF report generation
fastapi, uvicorn	Service layer and ASGI server

Table 3 - Python Dependencies and Their Roles

Installation and launch

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env          # then populate API keys and the model endpoint
uvicorn backend.main:app --reload --port 8000
```

On startup the application validates its configuration, prints any problems to the console, preloads the ATT&CK bundle, and serves the dashboard at *http://localhost:8000*. Directories that

TRACE writes to (*data/uploads, data/outputs, data/cache, data/mitre*) are created automatically at import time, so a fresh clone runs without manual preparation.
Evidence is prepared before upload. The analyst runs the appropriate Eric Zimmerman tool against each raw artifact and uploads the resulting CSV. TRACE deliberately does not invoke these tools itself: keeping extraction external means the conversion from raw hive to tabular data remains independently auditable, and TRACE's input is a file the analyst can inspect before submitting.

Configuration Details

All runtime parameters are centralised in *backend/config.py*, which loads *.env* at import time via *python-dotenv*. The loader is called with *override=True*, a deliberate deviation from the library default (see Configuration silently overridden by the environment for why). A *check_startup()* routine validates configuration before the server accepts requests and returns human-readable problems, printed to the console and exposed through */health*. Validating early matters operationally — without it, a missing model endpoint surfaces only after the analyst has uploaded evidence and waited through parsing and threat-intelligence lookups.

Parameter	**Value**	**Effect**
LOCAL_LLM_URL	*http://127.0.0.1:1234/v1*	LM Studio's OpenAI-compatible endpoint
LOCAL_LLM_TEMPERATURE	0.05	Near-deterministic sampling; keeps structured JSON well-formed
LOCAL_LLM_MAX_TOKENS	4096	Headroom for a reasoning block plus the JSON answer
LOCAL_LLM_BATCH_SIZE	10	Entries per Pass 1 request
PASS2_BATCH_SIZE	3	Entries per Pass 2 request (module constant)

Table 4 - Reasoning engine parameters

Parameter	**Value**	**Effect**
TI_TIMEOUT	15s	Per-lookup network timeout

TI_MAX_CONSECUTIVE_FAILURES	3	Failures before a source is abandoned for the run
VT_REQUESTS_PER_MIN	4	VirusTotal free-tier rate limit
METADEFENDER_REQUESTS_PER_MIN	10	MetaDefender rate limit
OTX_REQUESTS_PER_MIN	10	OTX rate limit

Table 5 - Threat intelligence parameters

Parameter	**Value**	**Effect**
TI_CONFIRM_THRESHOLD	5 engines	Detections required to treat a hash as confirmed malware
CONFIRMED_MALWARE_FLOOR	8.0	Minimum score for a confirmed hash
SEV_CRITICAL / SEV_HIGH / SEV_MEDIUM	7.0 / 5.0 / 3.0	Severity band boundaries

Table 6 - Scoring parameters

Externalising these values means a run is reproducible from its configuration rather than from its source code, and that a reviewer can vary a single parameter - the confirmation threshold, say - without touching the implementation.

Pipeline Implementation

Each stage of Figure 1 is implemented as a self-contained module, so that any stage can be replaced or tested without altering the rest of the pipeline.

Stage 1 - Artifact Parsing

Each artifact type has its own module: *amcache_parser.py*, *prefetch_parser.py* and *shimcache_parser.py*. All three build on pandas for CSV reading and python-dateutil for timestamp interpretation, and each exposes a single *parse(csv_path)* returning a list of dictionaries.

CSVs are read with *pd.read_csv(path, dtype=str, keep_default_na=False, on_bad_lines="skip")*. Each argument addresses a specific failure mode. Forcing *dtype=str*

prevents pandas inferring numeric types and corrupting SHA-1 values and timestamps into floating-point representations. Disabling *keep_default_na* stops pandas converting strings such as *"NA"* into *NaN*, which would then be indistinguishable from a genuinely absent field. Skipping bad lines rather than raising means one malformed row degrades coverage by a single record instead of aborting the investigation.

Column resolution uses a *COLUMN_ALIASES* dictionary mapping each internal field to the several names Eric Zimmerman's tools have used across versions, resolved case-insensitively once per file by *_resolve_columns()*. One parser therefore consumes CSVs from different tool releases without configuration.

Three helpers normalise every value. *_cell()* returns a cleaned string and maps the various spellings of emptiness (*"nan"*, *"n/a"*, *"null"*, *"none"*) to an empty string. *_to_bool()* interprets the several ways the tools spell a boolean. *_to_utc()* parses a timestamp with *dateutil*, attaches UTC when the value carries no timezone, and returns ISO-8601; an unparseable date returns an empty string rather than discarding the row, because the file remains evidence regardless of a malformed timestamp.

The Amcache parser additionally emits a boolean *publisher_recorded*, set from whether the source CSV contained a Publisher column at all - a different question from whether a given row has a publisher value. The significance of this distinction is discussed in the Absent data misread as negative data.

Stage 2 - Cross-Artifact Correlation

`correlation.py` has no external dependencies; it is pure Python over the dictionaries produced by Stage 1.

executable_stem() reduces a path to a matching key by normalising path separators, lowercasing, separating the parent directory from the final path component, and removing the extension with rsplit(".", 1). Splitting from the right once, rather than on the first separator, correctly handles names containing dots such as 7z2602-x64.exe. The resulting key is <normalised-parent-directory>:<filename-without-extension>. This lets the same executable at

the same location merge across artifacts while keeping same-named files in different locations separate — essential for detecting a masquerading copy such as C:\Users\analyst\Downloads\svchost.exe independently from the legitimate C:\Windows\System32\svchost.exe. When an artifact provides only a bare filename, the directory is recorded as unknown, so filename-only evidence can still be correlated.

build_index() groups rows by stem into an evidence structure recording which artifacts observed it, whether execution is proven, the highest run count, merged execution timestamps, every distinct path, and a bounded sample of loaded modules. The sample is capped at twenty entries per row: enough to reveal DLL side-loading without expanding the prompt beyond what a small model can attend to.

collapse_to_executables() performs the merge. Rows sharing a stem are sorted by a source_priority map placing Amcache first, because it is the only artifact carrying both a SHA-1 hash and signing metadata, and the highest-priority row becomes the base of the merged record. Identity fields absent from the base are back-filled from whichever row in the group possesses them. Two boolean fields require bespoke merge logic because their meaning is asymmetric: is_os_component is true if any source asserts it, and publisher_recorded is true if any source CSV carried the column. Execution evidence is combined by taking the maximum run count and the union of run timestamps.

evidence_for() renders the correlation result as short natural-language statements rather than numeric scores — "Execution PROVEN by Prefetch", "Executed 14 times (repeated use)", "Seen at 3 different paths". This is deliberate: the statements are written into the Pass 2 prompt as context, leaving the weighing of that evidence to the reasoning stage rather than encoding a weight here.

Stage 3 - Threat Intelligence Corroboration

Three client modules - *virustotal.py*, *metadefender.py*, *otx.py* - share a common structure and are coordinated by *threat_intel.py*. All use requests over HTTPS.

Each implements *_lookup_one()*, returning a result dictionary on success and None on network failure. The distinction is central to correctness: an empty or not-found response is a legitimate verdict and is cached, whereas a network failure is not a verdict and must never be cached. The Network failures cached as verdicts section describes what happened before this contract was enforced.

Each client also maintains a *consecutive_failures* counter. After *TI_MAX_CONSECUTIVE_FAILURES* failures the source is abandoned for the remainder of the run, with a message naming the source and the number of hashes left unchecked. Without this, an unreachable service costs *TI_TIMEOUT* seconds for every hash in the case - sixty hashes against a dead endpoint stalls an analysis for a quarter of an hour to produce results that were never going to arrive. Rate limiting is enforced per source through a minimum interval derived from a configured requests-per-minute value, and responses are cached to JSON under *data/cache/*. *threat_intel.py* exposes four functions used elsewhere: *lookup_all()* to query every unique hash, *fields_for()* to retrieve a flattened result, *has_signal()* used by the Selection Gate, and *summarise_for_ai()*, which reduces roughly fifty raw fields per service into a short curated summary. Summarisation matters because passing raw API responses into the prompt would bury the detection ratio - the single most decision-relevant figure - beneath scan metadata.

Stages 4 and 5 - The Reasoning Engine

local_llm.py communicates with LM Studio through the openai Python client pointed at a local base URL, exploiting LM Studio's OpenAI-compatible API. Using the standard client rather than raw HTTP means the same code can address any OpenAI-compatible endpoint.

run_pass1(rows) iterates the collapsed records in batches of *LOCAL_LLM_BATCH_SIZE*. The identifier given to the model for each entry is the row's index in the input list, and the model echoes it back, so results map home without offset arithmetic. Returned entries are validated before being retained: the echoed index must be an integer within

range, the proposed technique passes through *mitre.validate()* immediately so a fabricated identifier never propagates, and the score is clamped to 0–10. Pass 1 returns only the entries the model chose to flag.

run_pass2(candidates) operates on the candidate list in batches of three. The reduced batch size reflects that each Pass 2 record carries substantially more context - its own first-pass verdict, the curated antivirus summary and the correlation statements - so fewer records fit before the model's attention is diluted. Unlike Pass 1, Pass 2 returns a verdict for every candidate including those it judges benign, because the orchestrator needs an explicit final score for each. Where Pass 2 returns no valid technique, the Pass 1 technique is retained as a fallback.

Both passes route through a single *_call()* function, the only place in the pipeline aware of which engine is in use. Everything above it - prompt construction, batching, parsing, validation, scoring - is engine-agnostic.

Prompt Construction and Constraint

Prompt text lives in *ai_engine.py* rather than being embedded in the code that sends it, so the instructions given to the model are reviewable as a single artefact. Four templates are defined — a system and a user template per pass — together with a shared *SEVERITY_GUIDANCE* block injected into both system prompts by *build_pass1_system_prompt()* and *build_pass2_system_prompt()*. Sharing the guidance is necessary for consistency: were the scoring rules present in only one pass, the same file could receive different scores depending on which pass assigned them.

Division of responsibility between templates: The system template carries standing instructions - the analyst role, what evidence is and is not available at that stage, the judgement rules, the technique menu, and the output-format constraint. It is built once per pass and reused across every batch. The user template carries the work order for one batch: the entries themselves and the required JSON schema. This keeps the invariant instructions stable across a run while only the payload changes.

Rules rather than examples: *SEVERITY_GUIDANCE* opens by instructing the model to judge on what a file does and where it came from rather than on whether it recognises the product name. This inverts the failure mode a language model exhibits by default — treating

unfamiliar names as suspicious and familiar ones as safe — which is precisely the behaviour adversaries exploit by naming malicious binaries after legitimate system components. Evidentiary weight is placed on placement, signing status and capability instead.
Placement is expressed as three categories rather than a list of known-good directories: an operating-system or vendor-managed namespace; installer staging identifiable by GUID-formatted folder names; and general-purpose user-writable locations. Enumerating specific paths was attempted first and abandoned, because Windows relocates its own components between releases — Defender's executables run from a versioned directory under ProgramData rather than Program Files, and update components appear under directories introduced in recent feature updates. A hardcoded list flags each such relocation as tampering. Categorising the kind of location generalises across Windows versions.

Distinguishing absent data from negative data: *format_pass1_entry()* renders a row as compact JSON containing only decision-relevant fields, applying the *publisher_recorded* flag from Stage 1 - Artifact Parsing . A file with a publisher shows it; a file whose source recorded publishers but had none is labelled explicitly as unsigned; a file whose source never recorded publishers is labelled unknown. The guidance then states that only the first constitutes evidence. The same principle is applied throughout: fields with no value are omitted from the JSON entirely rather than sent empty, so missing data reads as absence rather than as a negative finding.

Output contract and parsing: Both user templates specify an exact JSON schema and require a bare array with no surrounding prose. extract_json_array() nonetheless handles three deviations observed consistently in small local models: a <think>...</think> reasoning block preceding the answer, stripped by regular expression; the array wrapped in Markdown code fences, matched and unwrapped; and explanatory prose surrounding the array, for which the parser falls back to slicing between the first [and the last]. Treating output malformation as expected rather than exceptional was necessary for the pipeline to complete reliably.

Stage 6 - Attribution Validation

mitre.py loads the ATT&CK Enterprise STIX bundle from a local copy at startup, parsing approximately fifty megabytes of JSON once so no analysis pays the cost. Preloading occurs in

the FastAPI lifespan handler, and a load failure is logged as a warning rather than preventing the server starting.

Grounding and validation are implemented separately. *prompt_menu()* renders a curated dictionary of forty execution-observable techniques into a terse menu inserted into both system prompts; the menu is generated from the loaded catalogue rather than hardcoded, so it cannot drift from the authoritative data. *validate()* then checks every returned identifier against the full parsed catalogue rather than against the menu, so an identifier the model produced from memory instead of from the menu is still verified. Unresolvable sub-technique identifiers fall back to their parent; anything unrecognised is rejected and appended to a hallucination list reported in the run metadata. Validation runs twice — once per pass — because Pass 2 may revise the attribution and its answer needs the same protection.

Stage 6b - Deterministic Scoring

scoring.py is deliberately small. *severity_for()* maps a score to a band using thresholds read from configuration. *finalise()* clamps the model's score to 0–10, queries *threat_intel.is_confirmed_malware()* against the configured engine threshold, and raises the score to *CONFIRMED_MALWARE_FLOOR* where the file is confirmed and the model scored it lower. The function contains no path that lowers a score.

An earlier implementation computed a weighted average of the model's score and the detection ratios. This was removed because Pass 2 is already shown those ratios before scoring, so blending them afterwards counts the same evidence twice. Each of the three branches produces a plain-language explanation string stored on the finding and printed in the report, recording whether the score is the model's own or was raised by the constraint.

Stage 7 - Reporting and Delivery

report.py builds the PDF with **ReportLab**, using *SimpleDocTemplate* with a shared stylesheet and a common table style so every section renders consistently. The document contains an executive summary, the risk-sorted findings table with the model's reasoning, the execution evidence established by correlation, the indicators of compromise, the chain-of-custody record and a methodology statement. The executive summary reports the count of entries admitted to Pass 2 solely by threat intelligence, making the Selection Gate's contribution visible in the deliverable itself.

orchestrator.py sequences the seven stages, holds the Selection Gate logic, and assembles run metadata - entry counts, hashes checked, Pass 1 and Pass 2 counts, intelligence-only reviews, MITRE validation results and correlation statistics. *main.py* exposes the service through FastAPI served by Uvicorn, with *POST /analyze* accepting the three CSV uploads and the chain-of-custody fields, and *GET /report/{id}* and *GET /export/{id}* for the deliverables. Exception handling maps failure classes to appropriate status codes: unparseable evidence returns 422, an unreachable reasoning engine or missing catalogue returns 503, anything else returns 500 with a logged traceback. The dashboard is a single static HTML page with vanilla JavaScript, requiring no build step.

Security Features

The implementation preserves several properties that follow from the threat model rather than from general good practice.

Hash-only external transmission: The only data TRACE sends outside the workstation is a SHA-1 hash, submitted to the three threat-intelligence services. No file content, file name, path, hostname or case identifier is transmitted. This preserves evidential confidentiality: a third party learns that someone asked about a hash, and nothing about the investigation. It is enforced structurally - the client modules accept a hash string and have no access to the record it came from.

Local inference: Both reasoning passes execute against a model running on the analyst's own hardware. Prompt content includes file names, paths and publisher strings, so routing it to a hosted model would transmit substantially more than the hash-only path above. Keeping inference local means the full prompt never leaves the machine.

No secrets in responses: The */health* endpoint reports whether each API key is configured as a boolean, never the key itself. Keys are read from *.env*, which is excluded from version control, and appear in no log line or error message.

Input validation before processing: Every upload is checked for a *.csv* extension in both *main.py* and *orchestrator.py* before any parsing occurs, and rejected with a message directing the

analyst to the correct pre-processing tool. Parsers read with *dtype=str*, so no cell content is evaluated or type-coerced.

Offline deliverable generation: ReportLab renders the PDF with no external fonts, stylesheets or network calls, so the report can be produced on an air-gapped workstation.

Cache contents: The threat-intelligence caches store hashes and detection results only; they contain no path, host or case data, so the cache files carry no evidential sensitivity if copied.

Attribution grounding: The closed-menu design and post-hoc validation of ATT&CK identifiers (Stage 6 - Attribution Validation) prevent fabricated attribution reaching a report that may be used to justify an operational response.

Implementation Challenges

Five problems required diagnosis during development. Each is recorded here because the resolution shaped the implementation.

A capacity ceiling on prompt length

An expanded revision of *SEVERITY_GUIDANCE* was written to address a set of false positives, growing the Pass 1 system prompt to approximately 8,400 characters. On a controlled test set the model's recall fell from thirteen of thirteen true positives to six. The longer instruction set degraded detection substantially while attempting to improve precision. Compressing the guidance to roughly 4,700 characters, preserving its rules while removing repetition, restored recall.

This is recorded as a finding rather than an incident. For a model of this size there is a practical ceiling on instruction length beyond which compliance degrades, so prompt engineering for small local models is constrained by a budget as well as by content. It also establishes a methodological caution: rule changes were subsequently validated against the model itself rather than by reasoning about the rules in isolation, because a rule set that is correct on paper may still not be followed.

Configuration silently overridden by the environment

The application repeatedly connected to a model endpoint that appeared nowhere in *.env*. Diagnosis proceeded through several false leads - stale file content, browser caching of the

/health response, and the presence of two project directories with separate virtual environments - before the cause was identified: *python-dotenv* does not by default replace a variable already present in the operating system environment, so a stale Windows user variable took precedence over the configuration file indefinitely.

The loader now runs with *override=True*, making *.env* authoritative. Additionally, the value of the relevant environment variable is captured before loading and compared afterwards, so *check_startup()* reports any variable that would previously have shadowed the file. Two secondary fixes followed: *Cache-Control: no-store* headers on */health*, because a cached response from a diagnostic endpoint is actively misleading, and *flush=True* on startup prints, because uvicorn's reloader buffers subprocess output and made a slow startup appear to hang.

Absent data misread as negative data

A run against a real Amcache export produced high-severity findings against signed Microsoft components, described in the model's own reasoning as unsigned copies masquerading as system binaries. Inspection of the CSV header showed that AmcacheParser's *UnassociatedFileEntries* export contains no Publisher column at all. The parser resolved the missing column to an empty string, and the prompt formatter rendered every row as explicitly unsigned. The model was reasoning correctly from a false premise supplied by the pipeline. The fix threads a *publisher_recorded* boolean from the parser through the correlation merge to the prompt formatter, allowing three distinct renderings - publisher present, checked and absent, never recorded - with only the second treated as evidence. The general lesson is recorded in the methodology: where a data source may omit a field entirely, the distinction between absent and negative must be represented explicitly, because a reasoning engine cannot recover it.

Network failures cached as verdicts

Threat-intelligence lookups were observed returning clean results for hashes that should not have been clean. Direct inspection of the cache files found 25 of 148 OTX entries holding what had originally been network errors, stored in the same shape as a legitimate empty result. Once cached, such an entry marked a file clean on every subsequent run of that case, and no retry would ever occur.

The return contract of every client was changed so *_lookup_one()* returns *None* on network failure, structurally distinct from an empty result, and only non-*None* values are cached.

The poisoned entries were purged. A related change added the consecutive-failure circuit breaker described in Stage 3 - Threat Intelligence Corroboration, after it was observed that an unreachable service at a thirty-second timeout would stall a sixty-hash case for approximately half an hour.

A false forensic signal, identified by direct verification

An early revision treated a zeroed or future-dated PE compile timestamp as evidence of tampering. This produced thirteen false positives in a single run, all against signed Windows system binaries in their correct system directories with clean antivirus results, each justified with an implausible date - 2079, 2065, 1980, 2043.

Rather than adjust the threshold, the field was verified directly. Reading the PE header of the corresponding binary on a live, unmodified Windows installation reproduced the same anomaly, establishing that the field does not hold a timestamp on modern binaries: deterministic build toolchains write a hash of the binary's own contents into that header field, which decodes as an arbitrary date. The premise was false, not the model's reasoning from it.

The guidance was amended to treat a link-date anomaly as weak evidence, admissible only in corroboration with an independent signal - genuine unsigned status, a user-writable location, a name imitating another binary, or an actual antivirus detection - and never sufficient alone to justify a claim of tampering.

Functional Prototype Demonstration

The following describes a complete analysis of a real Windows 11 dataset, comprising Amcache, Prefetch and ShimCache exports containing known-malicious samples alongside ordinary system and application activity.

Submission: The analyst uploads the three CSV exports through the dashboard and supplies chain-of-custody metadata — case identifier, examiner name, acquisition method and image hashes. Extensions are validated before processing begins.

Parsing: Each parser reports the number of entries recovered from its file. Timestamps are converted to UTC at this point, so all subsequent comparisons are on a single timeline.

Correlation: The console reports the reduction from artifact rows to unique executables — a program recorded in all three artifacts collapses to a single record carrying the union of what

the three observed. Files present in more than one artifact and files with proven execution are counted and retained in the run metadata.

Corroboration: Every unique hash is queried against the three services, with cached results reported separately from fresh lookups.

Pass 1: Records are submitted in batches of ten. The pass reports how many entries it flagged out of the total submitted.

Selection Gate: The orchestrator reports the number of entries advancing to Pass 2, and how many of those advanced solely because threat intelligence found a signal the first pass did not. In evaluation runs this figure was consistently non-zero, demonstrating that the disjunctive gate recovers findings a single-pass design would have discarded.

Pass 2: Candidates are submitted in batches of three with their antivirus and correlation evidence attached. Each returns a final score, a technique attribution and a written justification.

Validation, scoring and reporting: Technique identifiers are validated against the ATT&CK catalogue, the confirmed-malware floor is applied where applicable, and findings are rendered to PDF, CSV and JSON.

Outcome: Against a dataset containing nine independently confirmed malicious files, the prototype identified eight. The one missed sample and the false positives observed in the same run are analysed in the Results section, together with the changes made in response.

Experimental Assessment

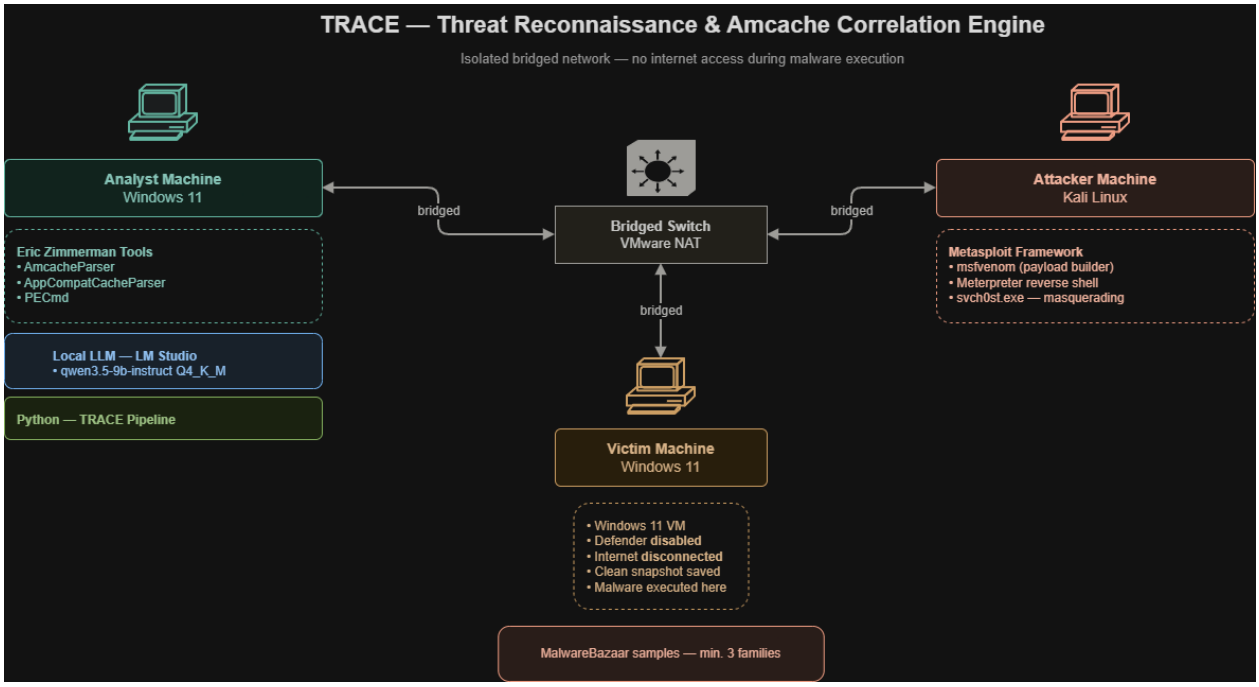


Figure 2 - Isolated Bridged Network Topology

Experiment 1: Reasoning Engine Substitution — Comparative Detection Performance of Two Language Models Across Two Independent Malware Datasets

Objective

TRACE's architecture isolates the reasoning engine behind a single dispatch function. Prompt construction, batching, cross-artifact correlation, threat-intelligence corroboration, attribution validation and deterministic scoring are all engine-agnostic, which means the language model can be substituted without altering any other component of the pipeline. This experiment exploits that property to ask the question the design was built to answer: when TRACE correctly identifies malware, is that because of the AI model, or because of the fixed rules built around it?

Two models are evaluated against two independently constructed datasets. The specific objectives are as follows. First, to establish comparative detection performance by measuring precision, recall and F1 for each model on each dataset, using an identical pipeline configuration

throughout. Second, to determine whether model capability is the limiting factor — if both models converge on similar results, the deterministic scaffolding (correlation, corroboration, the Selection Gate and the confirmed-malware floor) is carrying the detection performance, and if they diverge sharply, the reasoning engine is the dominant variable. Third, to isolate reasoning from lookup — each dataset contains a locally crafted Meterpreter executable that exists in no threat-intelligence database, meaning these samples cannot be detected by corroboration and can only be caught by reasoning over artifact metadata, which discriminates between models more sharply than any catalogued sample can. Fourth, to test whether the model's detection is genuinely contextual or superficially name-based — a sub-experiment renames the Meterpreter payload to innocuous names (*cloud.exe* and *cake.exe*) and re-runs the pipeline, testing whether the model still flags them when the filename itself provides no signal at all. Fifth, to test generalisation across datasets — running both models against two datasets built from different malware samples establishes whether a model's performance is a property of the model or an artefact of one particular sample set.

ID	Model	Execution
M1	Qwen 3.5 9B (Q4_K_M)	Local — LM Studio, RTX 3070
M2	Gemini 3.1 Flash Lite (medium thinking)	Hosted — Google API

Table 7 - Models Under Test

ID	Composition
D1	A set of malware samples obtained from MalwareBazaar, executed on a controlled Windows host, plus one locally crafted Meterpreter payload
D2	A disjoint set of MalwareBazaar samples, executed on the same host configuration, plus a second, newly generated Meterpreter payload distinct from D1's. Additionally, the Meterpreter payload was renamed to cloud.exe and cake.exe and included alongside the

	original to test whether detection survives a name change designed to remove the most obvious textual signal.

Table 8 - Datasets Under Test

Each dataset comprises the three artifact CSV exports (Amcache, Prefetch, ShimCache) recovered after execution, and therefore contains both the malicious samples and the ordinary system and application activity of a real Windows host.

Procedure

Ground truth was established before any model was run, and every executable in the population was labeled, not just the malicious ones. This is the single most important methodological requirement of the experiment, since a confusion matrix cannot be constructed without knowing the true class of every unique executable in the dataset. For each dataset, the pipeline's parsing and correlation stages were run first, and the collapsed list of unique executables was exported as the population — the set of items each model would classify. Each known-malicious sample was labeled as such, and every remaining executable (standard Windows components, installed applications, background services) was labeled as benign.

Both models were then run against both datasets independently, producing four primary pipeline runs: Qwen on D1, Qwen on D2, Gemini on D1, Gemini on D2. For each run, the full pipeline was executed end to end with identical configuration — the same correlation settings, the same threat-intelligence sources, the same scoring parameters — so that the language model was the only variable being changed between runs.

For the filename-substitution sub-experiment, the Meterpreter payload in Dataset 2 was renamed from *test_meterpreter.exe* to *cloud.exe* and *cake.exe* — names chosen specifically because they carry zero forensic signal on their own. These renamed copies were placed in user-writable directories (AppData and the Public folder) and processed through the same pipeline runs. If a model flags *test_meterpreter.exe* but misses *cake.exe* containing identical malicious

content, this would indicate the model is matching on the filename string rather than reasoning about the artifact's context. If both are flagged, it demonstrates that the model's reasoning is operating on deeper signals than filename alone.

Results

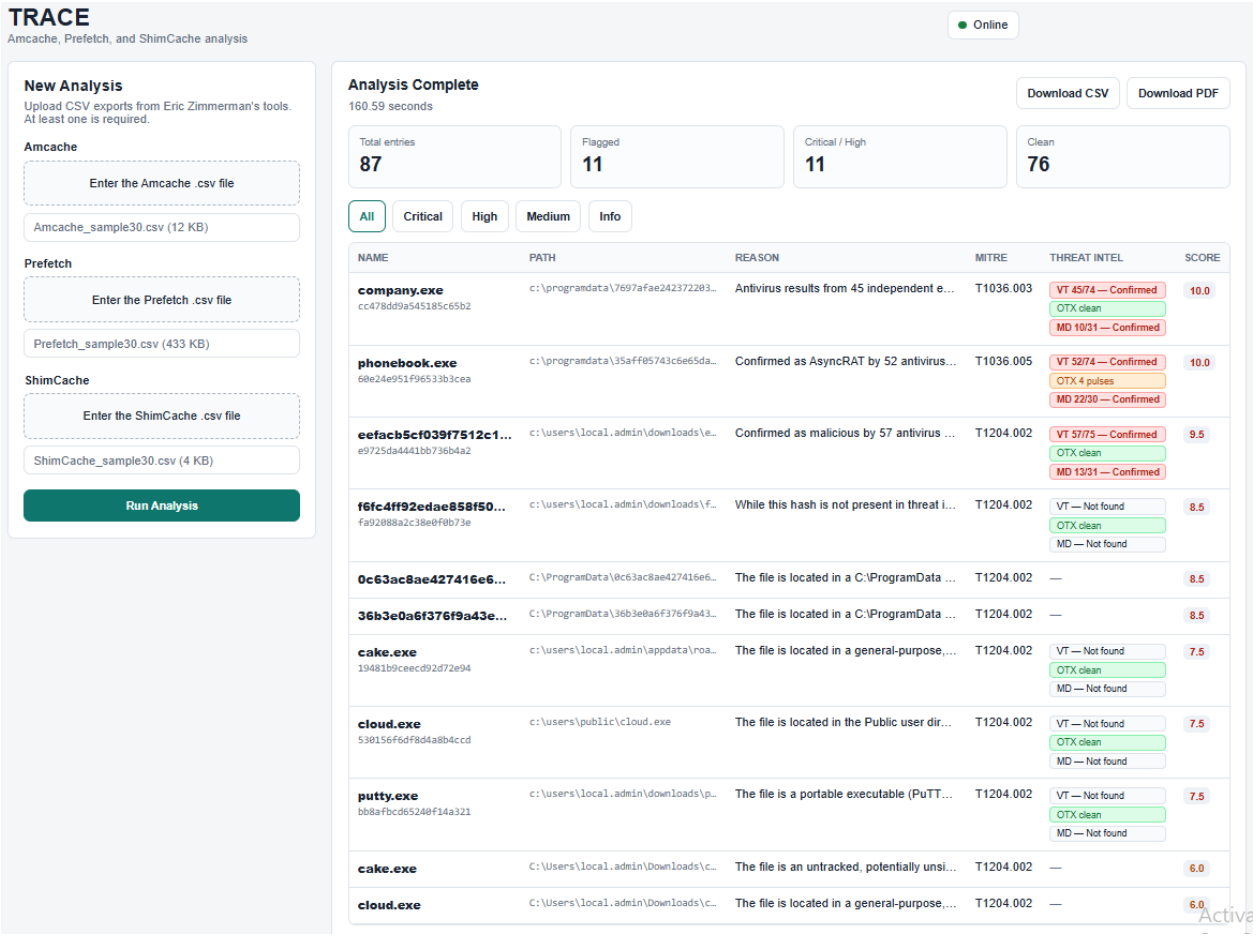


Figure 3 - Gemini D1 Dataset result

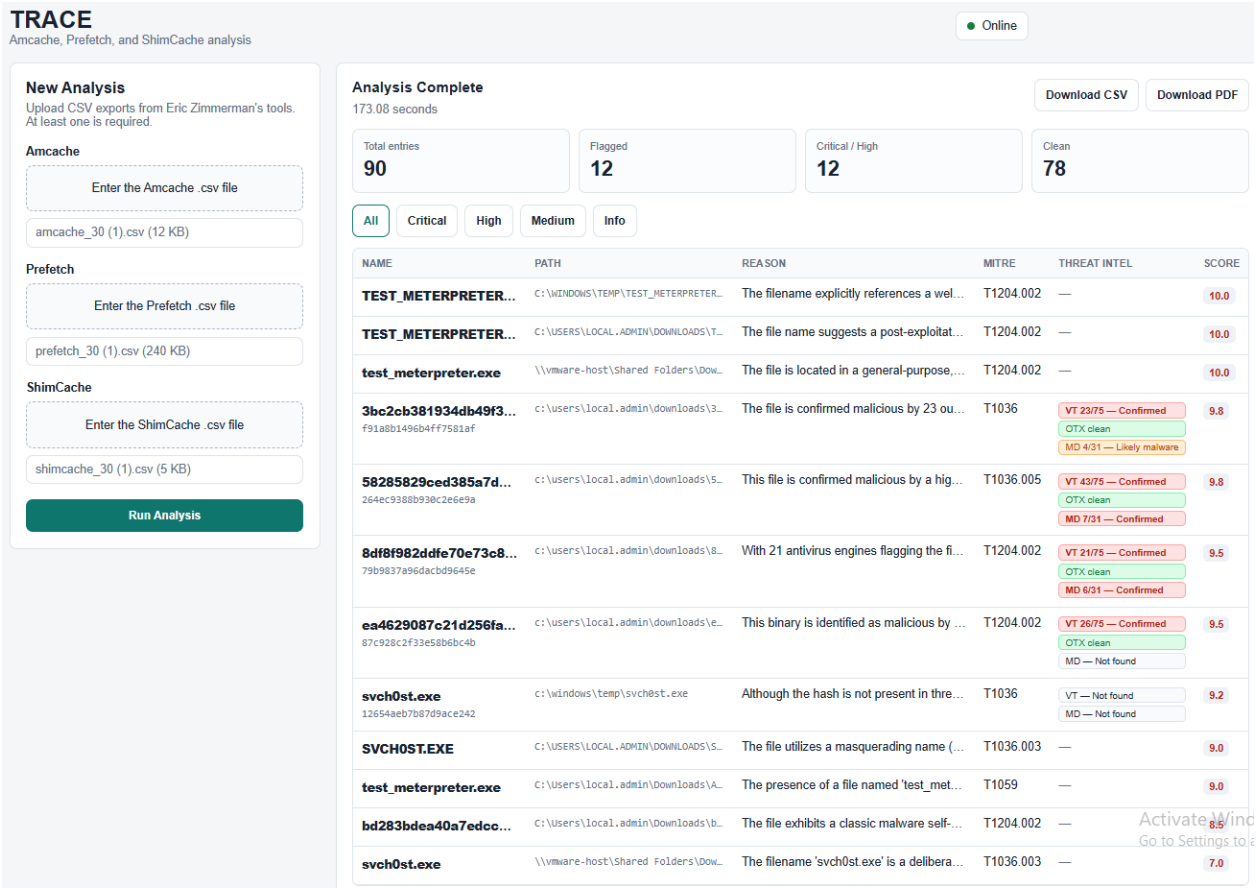


Figure 4 - Gemini D2 Dataset result

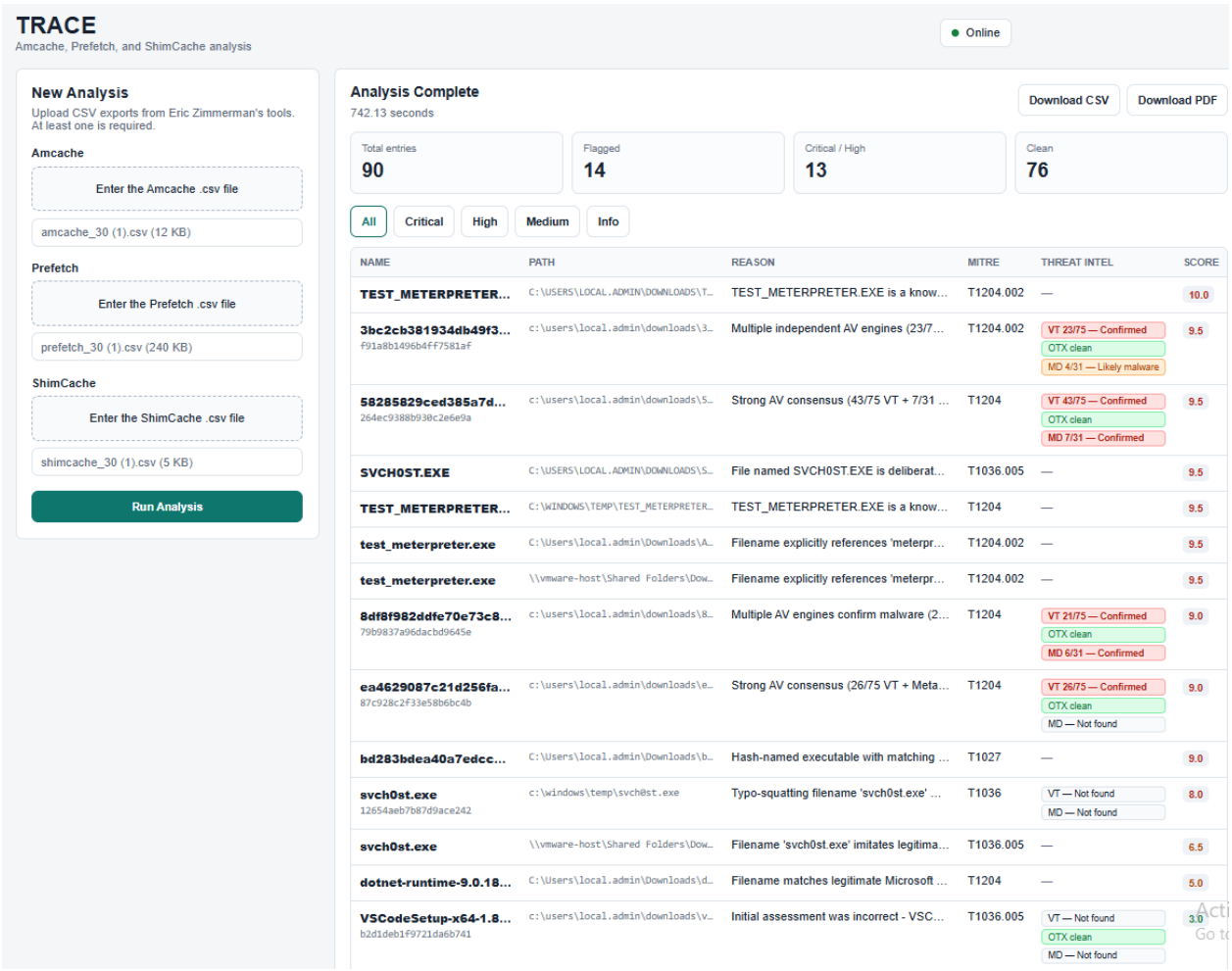


Figure 5 - Qwen D1 Dataset result

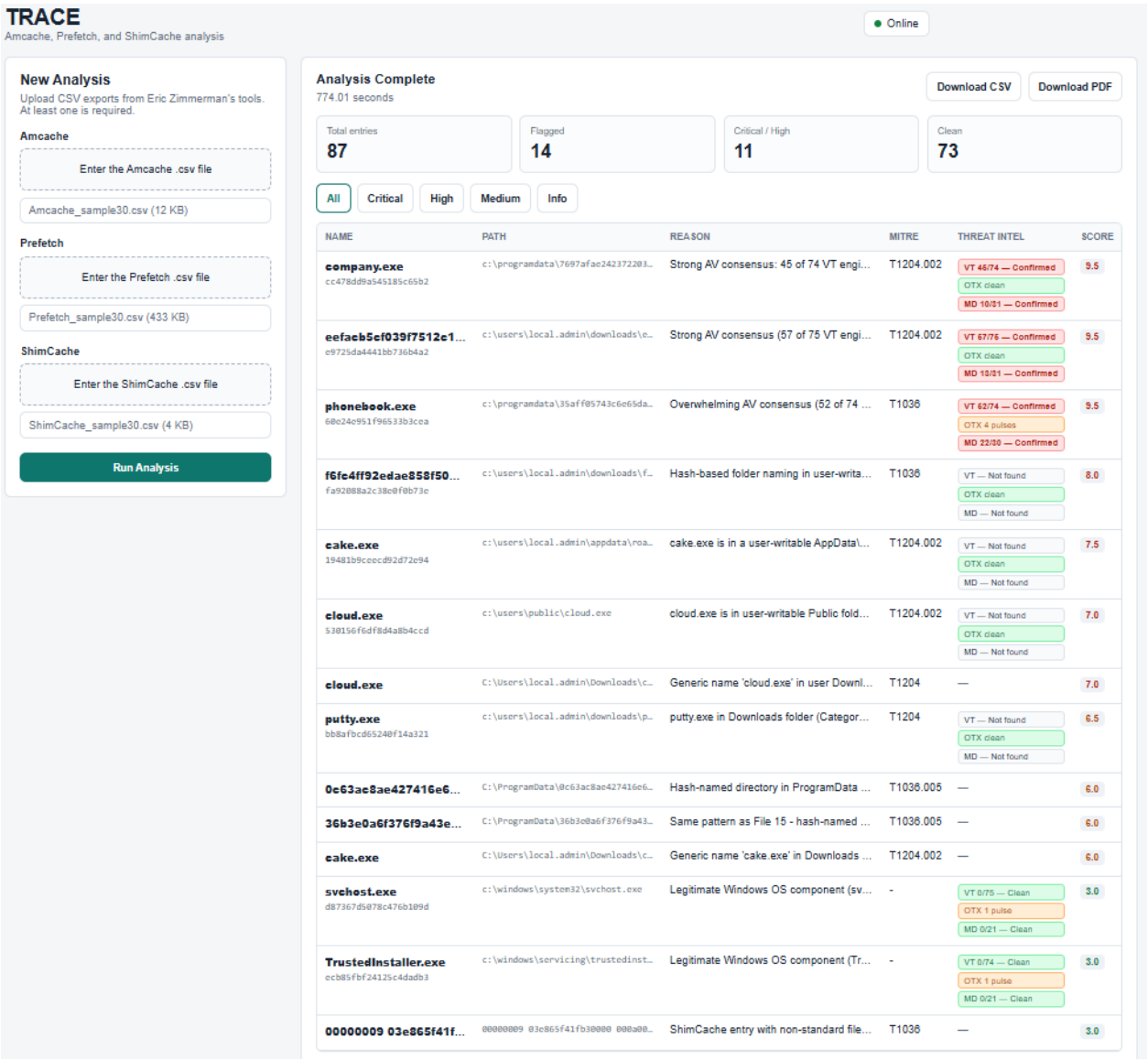


Figure 6 - Qwen D2 Dataset result

Graphs/Tables

Run	Precision	Recall	FPR	Accuracy	F1 Score
Gemini D1	7/7 = 1.00	7/9 = 0.78	0/67 = 0.00	74/76 = 0.97	0.87

Gemini D2	9/9 = 1.00	9/9 = 1.00	0/60 = 0.00	69/69 = 1.00	1.00
Qwen D1	8/8 = 1.00	8/9 = 0.89	0/67 = 0.00	75/76 = 0.99	0.94
Qwen D2	9/12 = 0.75	9/9 = 1.00	3/60 = 0.05	66/69 = 0.96	0.86
Gemini combined	16/16 = 1.00	16/18 = 0.89	0/127 = 0.00	143/145 = 0.99	0.94
Qwen combined	17/20 = 0.85	17/18 = 0.94	3/127 = 0.02	141/145 = 0.97	0.89

Table 9 - Comparison for Malware Datasets

In the filename-substitution sub-experiment, both cake.exe and cloud.exe — renamed copies of the Meterpreter payload carrying no malicious signal in their filename — were flagged by both models. Qwen assigned both a score of 7.5, citing their placement in user-writable directories and the absence of publisher metadata. Gemini similarly flagged both, citing the combination of file location and missing signing information. Neither model's detection relied on the original meterpreter filename string.

Analysis

The results show that both models achieved strong overall detection performance, but they failed in opposite directions, and this difference is the most important finding of the experiment.

Gemini produced zero false positives across both datasets, which gave it perfect precision of 1.00 on every run. However, it missed 2 of the 9 malicious files on Dataset 1, bringing its recall down to 0.78. This means Gemini is a conservative model: when it flags something, it is almost certainly correct, but it will sometimes let real threats pass without raising any alarm. For a forensic triage tool, this is the more dangerous failure mode, because a missed malicious file gives the analyst no indication that something was wrong at all.

Qwen showed the opposite pattern. On Dataset 1, it caught 8 of the 9 malicious files with zero false positives, which was a stronger result than Gemini on the same data. On Dataset 2, it caught all 9 malicious files and achieved perfect recall, but it also flagged 3 benign files as suspicious, producing a false positive rate of 5%. This means Qwen is a more aggressive model: it catches more real threats, but it occasionally raises alarms that an analyst would need to manually dismiss. For a triage tool specifically, this tradeoff is arguably more acceptable than Gemini's, since a false positive wastes a few minutes of analyst time while a false negative means a real threat goes undetected entirely.

Neither model's performance was consistent across both datasets. Gemini scored a perfect 1.00 F1 on Dataset 2 but only 0.87 on Dataset 1. Qwen scored 0.94 on Dataset 1 but dropped to 0.86 on Dataset 2. This means each model's accuracy is partially dependent on the specific samples it encounters, not just the model's own capability. This is exactly why the experiment used two independent datasets rather than one — a single dataset would have shown Gemini as clearly superior or Qwen as clearly superior depending on which dataset happened to be chosen, and either conclusion would have been misleading.

The combined figures across both datasets give the most honest overall comparison. Gemini achieved an F1 of 0.94 with perfect precision (1.00) and a recall of 0.89. Qwen achieved an F1 of 0.89 with slightly lower precision (0.85) and slightly higher recall (0.94). This shows that neither model is categorically better. They represent two different points on the precision-recall tradeoff, and which one is preferable depends on whether the analyst prioritizes avoiding false alarms or avoiding missed threats.

The most important finding, however, is what both models have in common rather than where they differ. Both models achieved accuracy above 96% on every single run, and neither

model produced an F1 below 0.86 on any run. This degree of convergence — high performance from two architecturally different models, one running locally on consumer hardware and one running on hosted infrastructure — supports the conclusion that TRACE's detection performance is not primarily a property of which language model is plugged in. The deterministic scaffolding around the model — the Selection Gate, the threat-intelligence corroboration, the confirmed-malware score floor, the MITRE attribution validation — is doing substantial work to keep both models within a high-performance band, regardless of their individual reasoning differences. This is directly relevant to the educational objective of this project: it demonstrates that the engineering around the model matters at least as much as the model itself.

Both datasets included a locally crafted Meterpreter payload that exists in no threat-intelligence database and therefore cannot be detected by hash lookup or AV corroboration alone. Detection of these samples depends entirely on the model's reasoning over artifact metadata — file location, naming patterns, and execution context. Both models successfully detected these samples in at least one dataset, which confirms that the AI reasoning layer is contributing genuine detection value beyond what the deterministic threat-intelligence pipeline could achieve on its own.

The filename-substitution sub-experiment provides the most targeted evidence of this reasoning capability. The Meterpreter payload was renamed from *test_meterpreter.exe* to *cloud.exe* and *cake.exe*, names specifically chosen because they carry zero forensic signal on their own, and reprocessed through the pipeline. Both renamed payloads were still detected and flagged: Qwen assigned both a score of 7.5, and Gemini flagged both as well. This is a meaningful result because it demonstrates that neither model is simply pattern-matching on the filename *meterpreter*. Both are reasoning about the combination of file location (user-writable AppData and Public directories), missing or absent publisher metadata, and behavioral context to reach a suspicion judgment even when the name itself provides no useful signal. This directly addresses one of the concerns raised in the literature review about LLM-based detection: that models might rely on superficial textual cues rather than genuine forensic reasoning. In this case, the detection survived a name change specifically designed to remove the most obvious textual cue, which provides concrete evidence that the model's reasoning, while imperfect, is operating on deeper signals than filename alone.

Experiment Test 2 – Benign-Only False Positive Stress Test — Model Comparison (Qwen vs. Gemini)

Objective

Experiment 1 measured detection recall — whether each model catches real malware. This experiment tests the opposite failure mode: whether each model over-flags software that is genuinely clean. A dataset containing zero malicious files was constructed specifically so that every single flagged finding is, by definition, a false positive. This isolates the comparison to false positive behavior alone, with no detection credit muddying the result, and directly tests which model is more trustworthy for unattended or lightly-supervised triage, where a high false positive rate erodes analyst confidence in the tool regardless of how good its recall is elsewhere.

Procedure

The same benign dataset (*Benign_Amcache_sample30.csv, Benign_Prefetch_sample30.csv, Benign_ShimCache_sample30.csv*) was run through the full TRACE pipeline twice — once with Qwen 3.5 9B (local) as the reasoning engine, once with Gemini 3.1 Flash Lite (hosted). All other pipeline configuration was held constant: the same 110 raw artifact rows, the same correlation logic collapsing them to 71 unique executables, and the same 26 unique hashes checked against VirusTotal, MetaDefender, and OTX. Since the dataset is confirmed benign end to end, ground truth for every one of the 71 executables is "not malicious," making the false positive rate simply the flagged count divided by 71.

Results

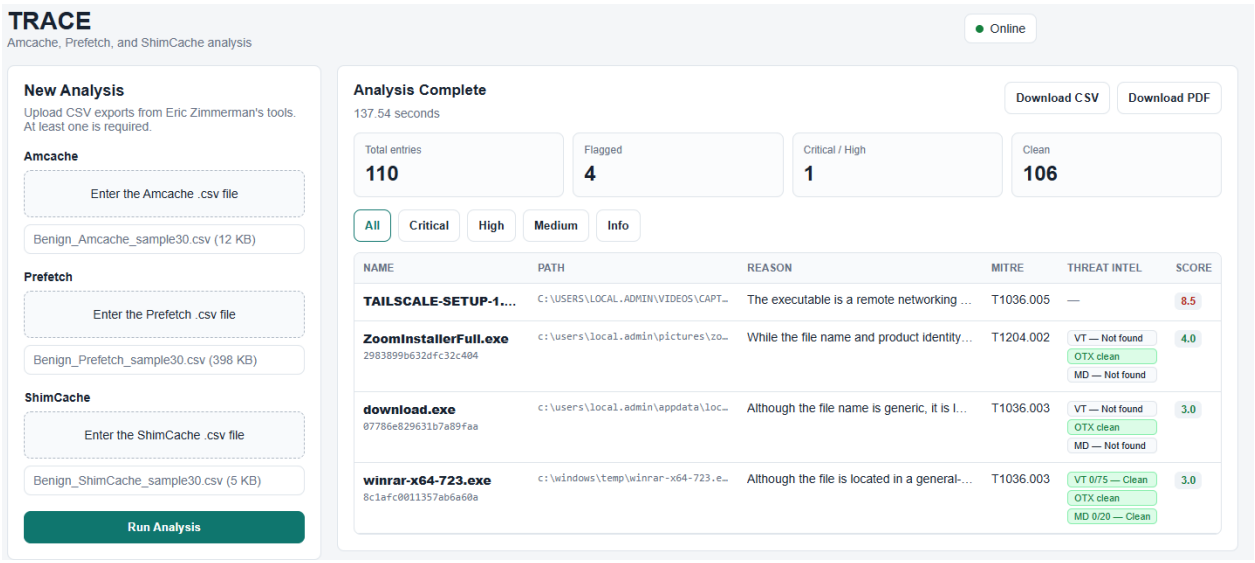


Figure 7 - Gemini Benign Dataset result

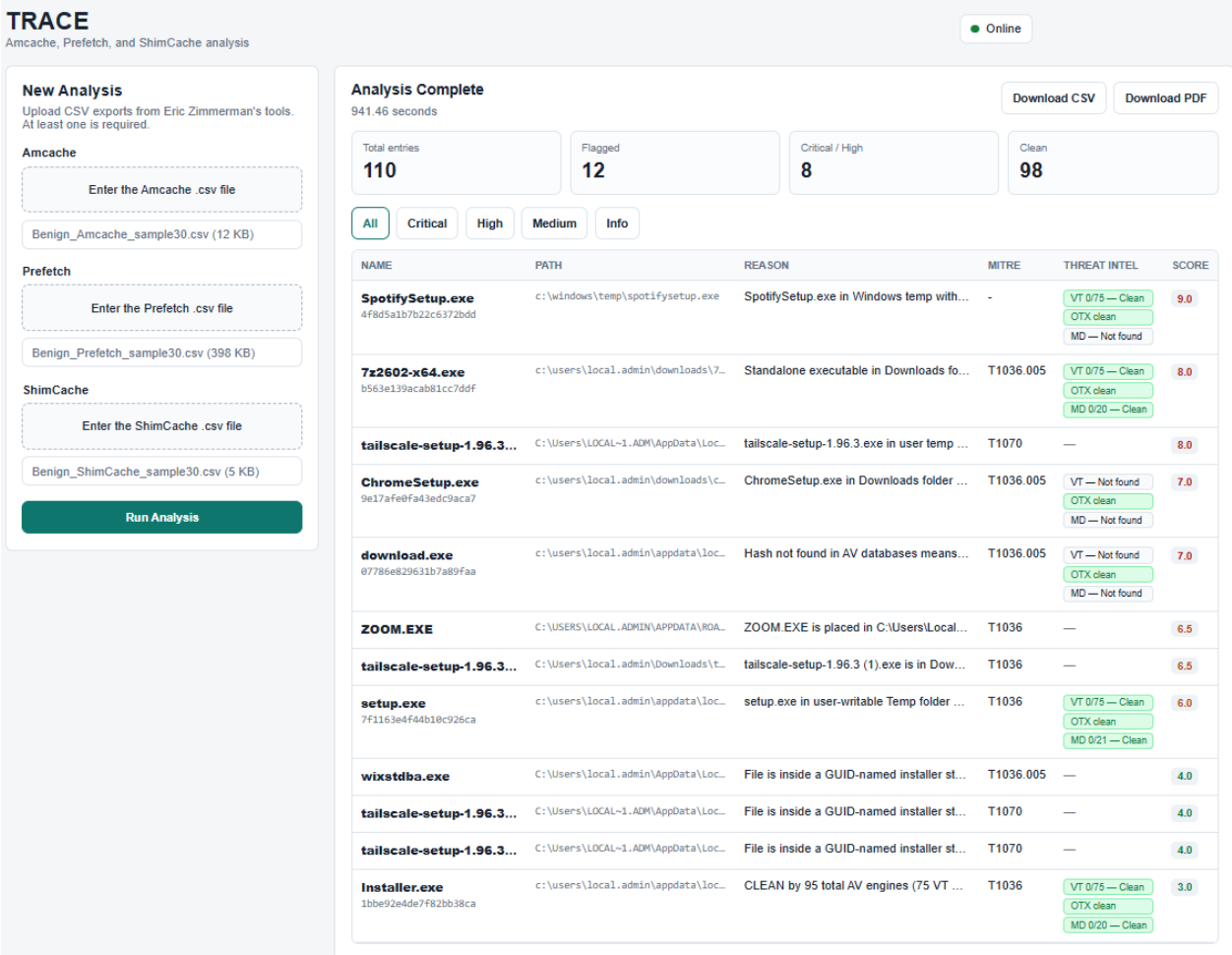


Figure 8 - Qwen Benign Dataset result

Graphs/Tables

Table 10 shows the pipeline's processing outcome for each model, from the initial metadata triage pass through to the final flagged count.

Model	Pass 1 Flagged	Sent to Pass 2	Pass 2 Verdicts	Final Findings	Processing Time
Qwen 3.5 9B	13 of 71	16 (3 rescued by TI)	9 of 16	**12**	941.46s
Gemini 3.1 + Flash Lite	9 of 71	12 (3 rescued by TI)	12 of 12	**4**	137.54s

Table 10 - Comparison table for Benign dataset results

Since the dataset is confirmed benign end to end, every one of these final findings is by definition a false positive. Table 11 converts these counts into standard classification metrics.

Model	False Positives	True Negatives	FPR	Accuracy
Qwen	12	59	12/71 = 16.9%	59/71 = 83.1%
Gemini	4	67	4/71 = 5.6%	67/71 = 94.4%

Table 11 - Metrics for benign dataset results

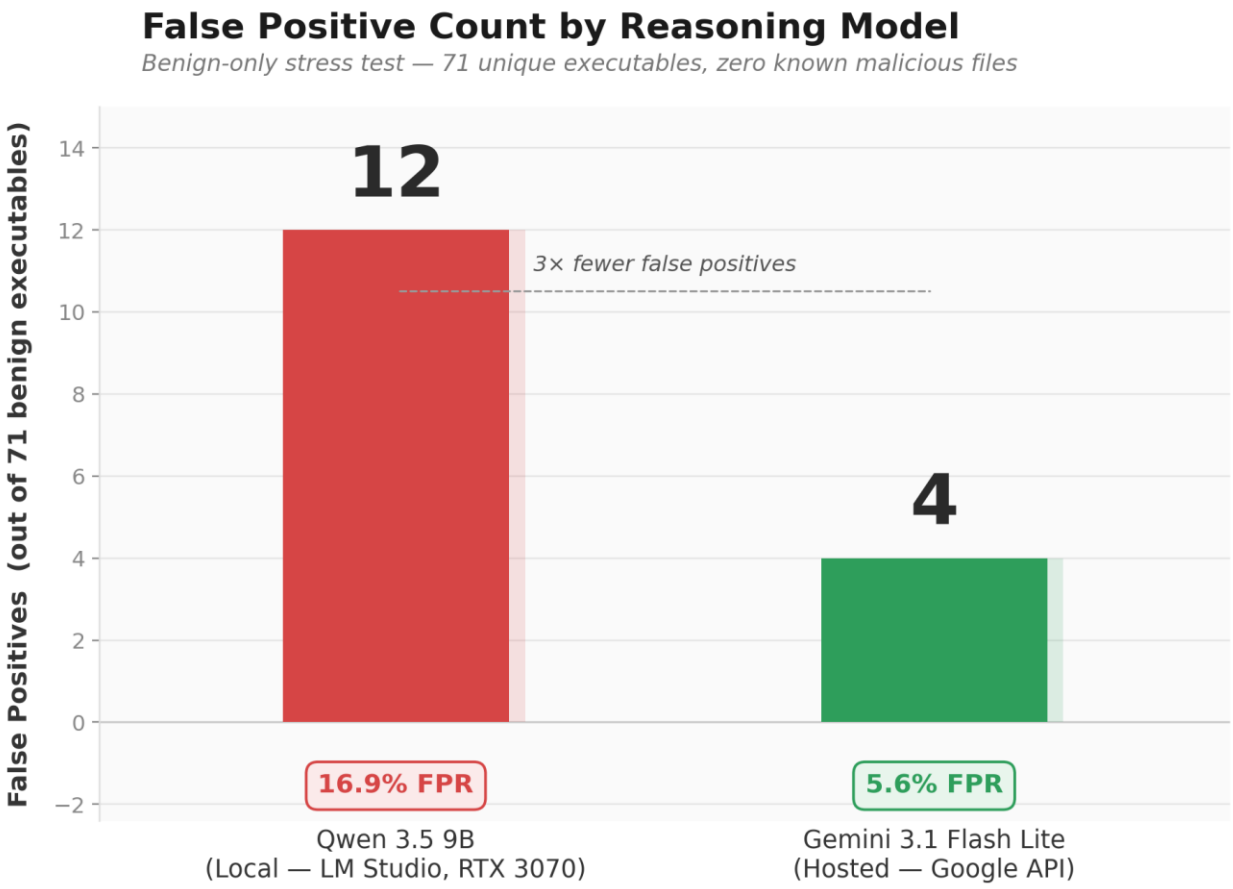


Figure 9 - Qwen & Gemini false positive rate graph

Model	***File***	***Score***	***VirusTotal***	***OTX***	***MetaDefender***
Qwen	*SpotifySetup.exe*	*9.0*	*0/75 — Clean*	*Clean*	*Not found*
Qwen	*7z2602-x64.exe*	*8.0*	*0/75 — Clean*	*Clean*	*0/20 — Clean*
Qwen	*tailscale-setup-1.96.3...exe*	*8.0*	*—*	*—*	*—*
Qwen	*ChromeSetup.exe*	*7.0*	*Not found*	*Clean*	*Not found*

Qwen	*download.exe*	*7.0*	*Not found*	*Clean*	*Not found*
Qwen	*ZOOM.EXE*	*6.5*	*—*	*—*	*—*
Qwen	*tailscale-setup-1.96.3...exe (2nd)*	*6.5*	*—*	*—*	*—*
Qwen	*setup.exe*	*6.0*	*0/75 — Clean*	*Clean*	*0/21 — Clean*
Qwen	*wixstdba.exe*	*4.0*	*—*	*—*	*—*
Qwen	*tailscale-setup-1.96.3...exe (3rd)*	*4.0*	*—*	*—*	*—*
Qwen	*tailscale-setup-1.96.3...exe (4th)*	*4.0*	*—*	*—*	*—*
Qwen	*Installer.exe*	*3.0*	*0/95 — Clean*	*Clean*	*0/20 — Clean*
Qwen total	***12 findings***				
Gemini	*TAILSCALE-SETUP-1...exe*	*8.5*	*—*	*—*	*—*
Gemini	*ZoomInstallerFull.exe*	*4.0*	*Not found*	*Clean*	*Not found*
Gemini	*download.exe*	*3.0*	*Not found*	*Clean*	*Not found*
Gemini	*winrar-x64-723.exe*	*3.0*	*0/75 — Clean*	*Clean*	*0/20 — Clean*

Gemini total	***4 findings***				

Table 12 - Findings from both models ran against the benign dataset

Gemini's 4 flagged findings, confirmed by the dashboard, were *TAILSCALE-SETUP-1...exe* (8.5), *ZoomInstallerFull.exe* (4.0), download.exe (3.0), and *winrar-x64-723.exe* (3.0). Notably, two of these — *download.exe* and *winrar-x64-723.exe* — were flagged despite clean verdicts from every threat intelligence source, including *winrar-x64-723.exe* showing an explicit 0/75 VirusTotal clean result and a 0/20 clean MetaDefender result.

Analysis

The results confirm the same directional pattern observed in Experiment 1: Qwen is the more aggressive, higher-false-positive model, and Gemini is the more conservative, lower-false-positive model. Because this dataset contains no malware at all, this experiment isolates that tendency cleanly, with no detection credit offsetting it — every flagged file here is unambiguously a mistake. Qwen made exactly three times as many false positive errors as Gemini on the identical dataset (12 versus 4).

A pattern worth highlighting is that even Gemini, the more conservative model, was not immune to disregarding clean threat-intelligence evidence. winrar-x64-723.exe was flagged despite an explicit 0/75 clean VirusTotal result and a 0/20 clean MetaDefender result — meaning 95 independent AV engines returned a clean verdict and Gemini still assigned it a score of 3.0. This shows that the tendency to under-weight confirmed-clean corroboration during the final reasoning pass is not unique to Qwen; it appears in both models, just to a much smaller degree in Gemini's case (4 such misjudgments versus 12).

Qwen's false positives remain considerably more severe both in count and in the confidence assigned to them. Several of Qwen's flagged files — *SpotifySetup.exe* at 9.0 and *7z2602-x64.exe* at 8.0 — were scored in the High or Critical band despite equally clean AV consensus, whereas Gemini's highest false-positive score in this run was 8.5 on a single file (*TAILSCALE-SETUP-1...exe*) and its remaining three false positives sat in the Medium band (3.0–4.0). This suggests that beyond simply flagging fewer benign files, Gemini's Pass 2

reasoning also assigns more conservative severity to the false positives it does produce — a second, compounding advantage on top of its lower raw count.

One structural observation supports the broader argument made throughout the evaluation section: both models were rescued into Pass 2 by exactly 3 threat-intelligence signals that Pass 1 alone had missed. Since the Selection Gate and TI corroboration logic are model-agnostic components of the pipeline, this identical number across two architecturally different models is expected, not coincidental — it demonstrates that the deterministic scaffolding around the reasoning engine behaves consistently regardless of which model is plugged in. What differs between the models is not this deterministic layer, but how each integrates the resulting AV evidence during its own final reasoning pass, and that difference is precisely where the gap between Qwen's 16.9% and Gemini's 5.6% false positive rate originates.

Finally, Gemini's Pass 2 in this run returned a verdict for all 12 of the 12 candidates it received, while Qwen's Pass 2 only returned verdicts for 9 of its 16 candidates. This is worth investigating further rather than treated as a footnote — it's unclear from the logs alone whether Qwen's remaining 7 candidates were silently dropped due to a parsing failure, a timeout, or a genuine "no finding" determination that wasn't logged the same way Gemini's was. If it's the former, this would be a reliability difference between the two integrations worth documenting separately from the accuracy comparison.

Experiment Test 3 – Gemini False-Positive Rate at Maximum Thinking Depth (Benign-Only Dataset)

Objective

Measure Gemini's false-positive rate at thinking_level=high in isolation, using a dataset containing zero malicious files — any entry that reaches a finding is by definition a false positive.

Procedure

Set GEMINI_THINKING_LEVEL=high in .env, restart, verify via /health. Run TRACE against a benign-only dataset (30 Amcache, 50 Prefetch, 30 ShimCache entries, no malicious samples planted). No other config changed.

Results

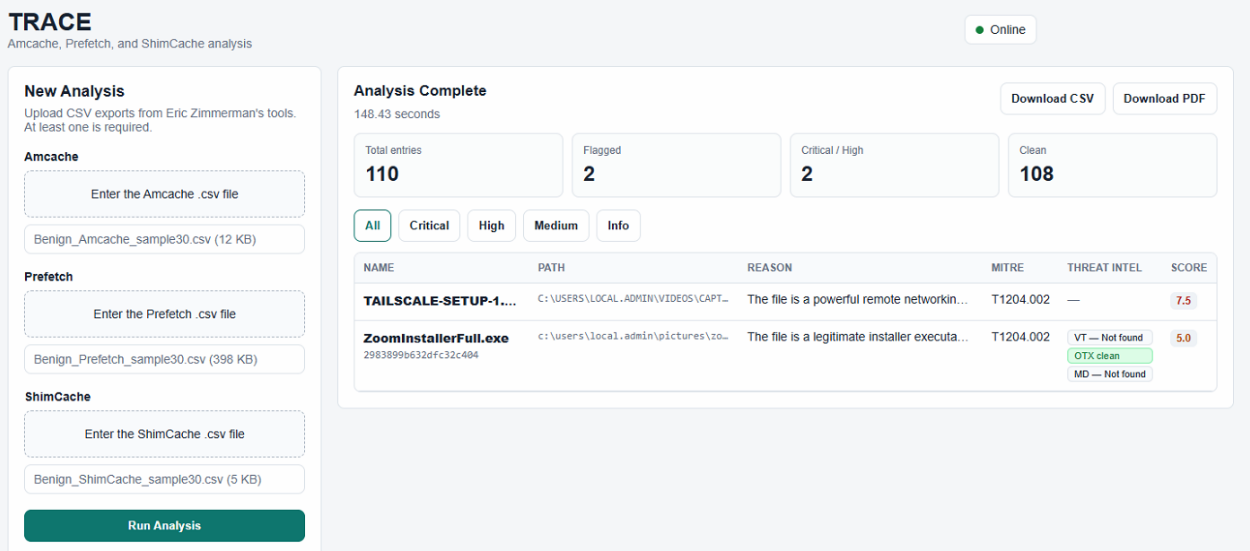


Figure 10 - Gemini ran against benign dataset with high thinking level

Analysis

At thinking_level=medium — the setting used for every other Gemini run in this report — the same benign dataset produced roughly 5 false positives. At thinking_level=high, that dropped to 2. The direction is consistent with what a larger reasoning budget should buy: more tokens spent per entry lets the model weigh corroborating evidence (execution history, correlation statements, absence of any TI signal) more carefully before committing to a verdict.

Results & Discussion

Summarize Results

Three experiments were conducted against the completed prototype, each isolating a different property of the design. The first substituted the reasoning engine itself Qwen 3.5 9B running locally against Gemini 3.1 Flash Lite running as a hosted service across two independently constructed malware datasets, each containing nine confirmed-malicious files among a majority of ordinary system and application activity. Both models remained above 96 per cent accuracy on every run. Gemini produced F1 scores of 0.87 on the first dataset and 1.00 on the second; Qwen produced 0.94 and 0.86 respectively, and neither model produced a single

false positive on the first dataset. A filename-substitution sub-experiment, in which the locally crafted Meterpreter payload was renamed to cake.exe and cloud.exe, found that both models continued to detect it, which rules out simple pattern-matching on the filename as the source of that detection.

The second experiment inverted the question, testing not whether the models catch malware but whether they over-flag software that is genuinely clean. A dataset containing no malicious files at all was constructed 110 artifact rows collapsing to 71 unique executables, with 26 unique hashes corroborated so that every finding produced is by definition an error. Qwen produced twelve false positives, a rate of 16.9 per cent; Gemini produced four, a rate of 5.6 per cent. Qwen's second pass also returned verdicts for only nine of the sixteen candidates admitted to it, a reliability gap distinct from the accuracy comparison and one the logs alone do not explain.

The third experiment held the dataset and the model constant and varied only the reasoning budget, re-running that identical benign dataset against Gemini with its thinking level raised from medium to high. The reduction appeared at both stages of the pipeline: the first pass flagged six entries rather than nine, and the final findings count fell from four to two.

Performance Analysis

The clearest result across the evaluation is that the two models fail in opposite directions, and that this difference matters more than the aggregate scores separating them. Gemini is conservative. It produced no false positives on either malware dataset, giving it perfect precision throughout, but it missed two of the nine malicious files on the first dataset and its recall suffered accordingly 0.78 on that dataset, 0.89 combined. Qwen is aggressive. It matched or exceeded Gemini's recall on every run, reaching 0.89 on the first dataset and 1.00 on the second for a combined 0.94, but paid for that with three false positives on the second dataset, reducing its combined precision to 0.85. Combined F1 favours Gemini slightly, 0.94 against 0.89, and read alone that figure would suggest a straightforward preference. It should not be read alone. A missed detection gives the analyst no indication that anything was wrong, whereas a false positive costs a few minutes of review and announces itself; for a triage tool intended to run with

limited supervision, the asymmetry between those two costs favours the model the aggregate score ranks second.

The benign-only experiment confirmed the same directional pattern on data where no detection credit could offset it, and did so with an unusually clean ratio: Qwen's false-positive rate was exactly three times Gemini's. More instructive than the ratio, however, was a single finding within it. Gemini flagged winrar-x64-723.exe at a score of 3.0 despite an explicit 0/75 clean result from VirusTotal and a 0/20 clean result from MetaDefender ninety-five independent engines returning a clean verdict, and the model assigning suspicion regardless. The tendency to under-weight confirmed-clean corroboration during the reconciliation pass is therefore present in both models, and differs between them in frequency rather than in kind.

The thinking-level result should be read against the second experiment's figures rather than an approximate baseline, since both runs consumed the identical dataset through an identical pipeline: four findings at medium, two at high. That the reduction appeared proportionally at both stages nine first-pass flags falling to six, four findings falling to two suggests the additional reasoning budget affects initial triage and final reconciliation to a similar degree rather than concentrating its effect in either. This rests on a single paired run, and the hosted model is not perfectly deterministic even at a fixed setting, so the result is best characterised as a direction consistent with expectation rather than an effect the evaluation established.

Runtime separated the two engines more sharply than accuracy did. Gemini completed the 71-executable benign run in 137.54 seconds against Qwen's 941.46, a difference of roughly seven times. That speed is real, but it is purchased by sending prompt content off the analyst's workstation, which is a security property rather than a performance one and is treated as such below.

Security Analysis

The corroboration layer behaved as designed throughout the evaluation. Only file hashes were transmitted across all three experiments no filename, path, hostname or case identifier left the workstation at any point and this property held identically regardless of which reasoning

engine was under test, because the threat-intelligence clients sit outside the reasoning dispatch entirely and have no access to the records their hashes came from.

The reasoning layer does not share that property equally across configurations, and the experiments made the difference concrete rather than hypothetical. The Qwen runs kept every prompt on the analyst's own hardware, including the file names, paths and publisher strings that prompt necessarily contains. The Gemini runs sent that same content to a third-party API, because substituting the reasoning engine means the full prompt must reach whichever endpoint is configured. The faster and more precise configuration is therefore also the configuration that transmits the richest evidence beyond the analyst's control. A deployment choosing between them is choosing between throughput and evidential confidentiality, not merely between two points on a detection-performance curve and for forensic work, that is not a trade the accuracy figures alone can adjudicate.

The attribution validation layer was exercised on every model call across all three experiments, since both passes route their proposed technique identifiers through validation before anything is retained. None of the three experiments tabulated the resulting rejection and fallback counts as a reported metric, which is a gap worth closing: the mechanism was active throughout and its output is already recorded in each run's metadata, but its effectiveness during these particular runs was never quantified with the rigour applied to detection performance.

Finally, the caching-integrity correction documented among the implementation challenges enforcing that a network failure is structurally distinct from a legitimate empty result, and that only the latter is ever cached was in place for all three experiments. Had it not been, the false-positive figures reported above would be unreliable rather than merely imprecise, since a single poisoned cache entry marks a file clean on every subsequent run of the same case and no retry ever occurs.

Compare with Existing Solutions

No head-to-head benchmark was run against Autopsy, KAPE, Velociraptor or any commercial triage platform, and the comparison offered here is accordingly architectural rather than empirical. What the results do support is the distinction drawn in the literature review.

Established platforms surface a verdict without exposing the reasoning that produced it, whereas every finding across these three experiments carries a written justification, a statement recording whether its score was the model's own or was raised by the deterministic floor, and in cases such as the winrar-x64-723.exe finding a visible record of the corroborating evidence the model was shown and evidently discounted. That visibility is not incidental to the evaluation; it is what made several of this project's more interesting observations available at all. Qwen's second-pass drop-off and Gemini's override of a clean multi-engine consensus were both identified by reading what the pipeline recorded about its own reasoning. A platform that returned the same verdicts without that record would have produced the same findings with no means of interrogating why.

Discussion and Interpretation

The question posed in the introduction, and restated as the first experiment's objective, was whether a small language model bounded on both sides by deterministic logic can contribute usefully to forensic triage. The evaluation answers this in two parts, and the two parts pull in different directions.

The first is that the deterministic scaffolding is doing most of the measurable work. Two architecturally dissimilar models, one quantised to fit within 8 GB of consumer video memory, the other a substantially larger model running on hosted infrastructure both remained above 96 per cent accuracy on every run despite reasoning visibly differently about the same evidence. That degree of convergence is difficult to explain except by the components that both configurations share: correlation collapsing each executable to a single assessment, unconditional corroboration of every hash, the disjunctive Selection Gate, and the confirmed-malware floor. The engineering surrounding the model constrains both toward a similar band of outcomes largely independently of the model's own disposition.

The second is that the reasoning layer is nonetheless not redundant with that scaffolding. Both models detected the locally crafted Meterpreter payload that exists in no threat-intelligence database and could therefore not have been recovered by corroboration under any configuration. More pointedly, both continued to detect it after it was renamed to cake.exe and cloud.exe, names chosen precisely because they carry no forensic signal. Detection survived the removal of the most obvious textual cue, which is direct evidence that the models are reasoning over

placement, signing status and execution context rather than matching a string and it addresses, on this dataset at least, one of the specific concerns the literature raises about applying language models to security triage.

The benign-only experiment tempers both conclusions. Both models erred at a non-trivial rate against data containing no malicious content whatsoever, and Gemini's decision to flag a file that ninety-five independent engines had cleared demonstrates that even the more conservative model will occasionally discount deterministic evidence placed directly in front of it. This is the sharpest illustration in the report of why the architecture withholds final authority over severity from the reasoning engine. It also exposes an asymmetry in the current design: the confirmed-malware floor exists to correct a model that under-reacts to unambiguous external evidence, but nothing in the pipeline performs the equivalent correction for a model that over-reacts to evidence pointing the other way. The failure mode the second experiment surfaced is precisely the one the constraint layer does not currently address.

The third experiment suggests that the position a configuration occupies on the precision–recall tradeoff is not fixed even once the model has been chosen. The same model, given a larger reasoning budget, moved further into territory it already occupied fewer false positives, greater conservatism without any corresponding measurement of whether recall was surrendered in the process. Establishing that is the natural continuation of this work rather than a question it answers.

Limitations

Technical Limitations

TRACE reasons entirely over metadata artifacts and hash-based reputation, and never inspects the bytes of the executable itself. There is no static analysis of PE structure, no unpacking, no string extraction and no behavioural sandboxing. Detection is therefore bounded by what execution artifacts and external reputation are able to reveal between them, and a malicious binary combining an unremarkable name, a plausible installation path and no threat-intelligence record could evade the reasoning passes and the Selection Gate simultaneously the

gate admits an entry on a signal from either source, so a file producing neither is never reviewed at all.

Small local models carry a genuine capacity ceiling on instruction length, established during development rather than assumed: an expanded revision of the severity guidance reduced recall from thirteen of thirteen true positives to six. That constraint was compressed around rather than eliminated, and any future addition to the rule set reintroduces the same risk. Prompt changes for a model of this size must therefore be validated against the model empirically rather than reasoned about in isolation, because a rule set that is correct on paper may still exceed what the model can reliably follow.

Both passes depend on evidence rendered as short natural-language statements. Anything not expressible in that form binary similarity, packing signatures, code reuse across related samples is invisible to the reasoning stage by construction, and no amount of prompt refinement can recover a signal the pipeline never extracted.

Data Limitations

Ground truth across all three experiments was established by the same team that built the detection logic. Known-malicious samples were planted deliberately and every remaining executable was labelled benign, with no independent audit of that labelling. For the benign-only corpus in particular, an unwanted program present in the sample set without the team's knowledge would be scored as a false positive rather than a correct detection, which would understate the tool's real precision rather than inflate it.

Population sizes were modest 76 and 69 unique executables in the two malware datasets, 71 in the benign corpus and each condition was run once. There are no repeated trials from which to estimate variance, which matters most for the thinking-level comparison, where a change from four findings to two rests on a single paired run against a model that is not perfectly deterministic between invocations.

The two malware datasets are also less independent than the experiment's framing implies. Both combine MalwareBazaar samples with a locally crafted Meterpreter payload

produced by the same team through the same process on the same host configuration. The generalisation observed between them is real, but it is generalisation across two similarly constructed datasets rather than across materially different threat scenarios.

Environmental Constraints

The local reasoning model was bounded by 8 GB of video memory, which forced the Q4_K_M quantisation level documented in the implementation. The results reported for the local model are properties of that specific configuration on that specific hardware, and a workstation able to host a larger or less aggressively quantised model might exhibit a different precision–recall profile entirely.

The hosted model's behaviour, availability and rate limits lie outside the project's control. Free-tier quota exhaustion was encountered directly during development, and both the model lineup and the default behaviour of a hosted service may change between this evaluation and any attempt to reproduce it.

Every experiment ran against a single controlled virtual machine snapshot per dataset. The results reflect that machine's particular artifact volume and software inventory rather than the baseline of a production fleet.

Threats to Validity

Internal validity is limited by the same team having defined both the detection logic and the ground truth against which that logic was scored, which admits the possibility that the classification boundary was drawn in a manner favourable to the tool's own design.

Construct validity is limited by how a false positive was operationalised. Any finding not appearing on the planted-malicious list was counted as an error, even where flagging the file represents defensible triage behaviour rather than a mistake. The benign-only findings illustrate the difficulty directly: SpotifySetup.exe, 7z2602-x64.exe and the several Tailscale installer entries are exactly the unfamiliar-but-legitimate installers the guidance instructs the model to reason past, and the boundary between an over-cautious flag and a wrong one is not as sharp as the metric treats it.

External validity is limited to a single Windows 11 virtual machine carrying a curated software inventory, and the results may not transfer to an enterprise environment with a different baseline, patch cadence or adversary tooling.

Statistical conclusion validity is limited by the single-run-per-condition design throughout. Every precision, recall and F1 figure reported is a point estimate without a confidence interval, and the thinking-level finding in particular should be treated as a direction warranting confirmation rather than an effect this evaluation demonstrated.

Conclusion and Future Work

Research Summary

This project set out to determine whether a small language model, deliberately bounded on both sides by deterministic logic, can contribute usefully to the forensic triage of Windows execution artifacts. TRACE was built to test that question rather than to deliver a production tool, and its architecture follows from it directly: three independent parsers normalise Amcache, Prefetch and ShimCache into a single representation in UTC; a correlation layer merges every observation of the same executable so that each program is judged once and never from partial metadata; a corroboration layer tests every unique hash against three independent vendors while transmitting nothing but the hash; and a two-pass reasoning stage, bounded by a disjunctive Selection Gate before it and by attribution validation and a one-directional scoring floor after it, produces the findings the analyst receives. Three experiments then evaluated that design against real malware, against constructed payloads catalogued nowhere, and against evidence containing no malicious content at all.

Key Findings

The dominant contributor to overall accuracy is the deterministic scaffolding rather than the choice of reasoning model. Two architecturally different models, one quantised and local and one substantially larger and hosted, both remained above 96 per cent accuracy on every malware run despite reasoning visibly differently about identical evidence — a convergence explicable only by the components they share.

Model choice nonetheless shifts the precision–recall tradeoff meaningfully, and neither model is categorically superior. Gemini is conservative, achieving perfect precision at the cost of recall for a combined F1-score of 0.94; Qwen is aggressive, achieving higher recall at the cost of precision for a combined 0.89. Which is preferable depends entirely on whether missed detections or false alarms carry the greater cost in a given deployment, and the aggregate F1-score ranking does not settle that question.

The reasoning layer contributes detection value that corroboration alone could not supply. Both models identified a locally crafted Meterpreter payload absent from every threat-intelligence database, and both continued to identify it after it was renamed specifically to strip away its only obvious textual cue — evidence that the reasoning operates on placement, signing and context rather than on filename recognition.

Neither model, however, is dependable enough to be trusted without the deterministic layers surrounding it. The benign-only stress test produced a substantial false-positive rate from both, and Gemini's decision to flag a file cleared by ninety-five independent engines demonstrates that even the more conservative configuration will override deterministic evidence placed directly before it. The confirmed-malware floor corrects the inverse error; the current design has no equivalent safeguard against this one.

Reasoning budget emerged as a further lever independent of model selection. Raising the hosted model's thinking level on an identical dataset reduced both first-pass flags and final findings, though on a single paired run that requires repetition before the effect can be considered established.

Several negative results carry value independently of TRACE itself: the capacity ceiling on instruction length for small local models, the unreliability of the PE compile-timestamp field as a tampering indicator now that deterministic build toolchains write a content hash into it, and the necessity of representing absent and empty data as structurally distinct states before handing either to a reasoning engine.

Contributions

The project contributes a working and fully documented triage prototype organised around a specific architectural pattern — a probabilistic reasoning core bounded on both sides by deterministic logic — and built so that every design decision and every failure encountered in reaching it remains open to inspection by someone learning the subject. It contributes an empirical comparison of a local and a hosted language model substituted into an otherwise identical pipeline, which isolates the model's contribution from the architecture's in a way the literature reviewed for this project rarely makes explicit. And it contributes a set of documented, reproducible negative results — the prompt-length ceiling, the false timestamp signal, the conflation of absent with empty data, and the caching failure that silently inverted verdicts — which transfer to any comparable project regardless of the specific model or artifacts involved.

Future Improvements

Ground truth should in future be established independently of the team building the detection logic, removing the internal-validity concern noted above, and each experimental condition should be repeated sufficiently to report confidence intervals rather than single-run point estimates. Static and binary-level analysis — PE section inspection, unpacking and string extraction — should be added as a fourth evidence source alongside metadata and reputation, since it addresses directly the signal-quality limitation that currently leaves a well-placed and unremarkable-looking binary undetectable by any existing source. Attribution rejection and fallback counts should be tabulated per run as a standard reported metric, given that the validation mechanism already executes on every model call and only its reporting is missing. And the thinking-level effect observed in the third experiment should be tested against the malware datasets rather than the benign corpus alone, to establish whether the reduction in false positives is genuine improvement or is being purchased with a corresponding loss of recall.

References

[1] K. Kent, S. Chevalier, T. Grance, and H. Dang, *Guide to Integrating Forensic Techniques into Incident Response*, NIST Special Publication 800-86, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2006. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-86.pdf

[2] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Rev. 2, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2012. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf

[3] International Organization for Standardization, *ISO/IEC 27037:2012 — Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence*, ISO, Geneva, Switzerland, 2012. [Online]. Available: https://www.iso.org/obp/ui/#iso:std:iso-iec:27037:ed-1:v1:en

[4] E. Zimmerman, "Eric Zimmerman's tools." [Online]. Available: https://ericzimmerman.github.io/

[5] K. Guðjónsson, "Mastering the super timeline with log2timeline," SANS Institute InfoSec Reading Room, GCFA Gold Certification paper, Jun. 2010. [Online]. Available: https://www.sans.org/white-papers/33438

[6] B. Carrier, "The Sleuth Kit and Autopsy: Open source digital forensics." [Online]. Available: https://www.autopsy.com/

[7] Kroll, "Kroll Artifact Parser and Extractor (KAPE)." [Online]. Available: https://www.kroll.com/kape

[8] Rapid7, "Velociraptor: Digging deeper — endpoint visibility and collection." [Online]. Available: https://docs.velociraptor.app/

[9] V. M. Álvarez, "YARA: The pattern matching swiss knife for malware researchers." [Online]. Available: https://virustotal.github.io/yara/

[10] SigmaHQ, "Sigma: Generic signature format for SIEM systems." [Online]. Available: https://github.com/SigmaHQ/sigma

[11] V. Jusas, D. Birvinskas, and E. Gahramanov, "Methods and tools of digital triage in forensic context: Survey and future directions," *Symmetry*, vol. 9, no. 4, art. 49, Apr. 2017, doi: 10.3390/sym9040049. [Online]. Available: https://www.mdpi.com/2073-8994/9/4/49

[12] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symposium*, Santa Clara, CA, USA, Aug. 2019, pp. 729–746. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury

[13] VirusTotal, "VirusTotal API v3 documentation." [Online]. Available: https://docs.virustotal.com/reference/overview

[14] M. Sebastián, R. Rivera, P. Kotzias, and J. Caballero, "AVclass: A tool for massive malware labeling," in *Proc. 19th Int. Symp. Research in Attacks, Intrusions, and Defenses (RAID)*, Paris, France, 2016, pp. 230–253. [Online]. Available: https://www.semanticscholar.org/paper/f74804eaf20b71da1ad2ebbbb429595c133459c8

[15] S. Zhu, J. Shi, L. Yang, B. Qin, Z. Zhang, L. Song, and G. Wang, "Measuring and modeling the label dynamics of online anti-malware engines," in *Proc. 2020 ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2020, doi: 10.1145/3372297.3420013. [Online]. Available: https://dl.acm.org/doi/10.1145/3372297.3420013

[16] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, *MITRE ATT&CK: Design and Philosophy*, The MITRE Corporation, McLean, VA, USA, Tech. Rep., rev. Mar. 2020. [Online]. Available: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[17] OASIS Open, "Structured Threat Information Expression (STIX) 2.1." [Online]. Available: https://oasis-open.github.io/cti-documentation/

[18] G. Gerganov et al., "llama.cpp: LLM inference in C/C++." [Online]. Available: https://github.com/ggml-org/llama.cpp

[19] LM Studio, "LM Studio — discover, download and run local LLMs." [Online]. Available: https://lmstudio.ai/

[20] G. Michelet and F. Breitinger, "ChatGPT, Llama, can you write my report? An experiment on assisted digital forensics reports written using (local) large language models," *Forensic Science International: Digital Investigation*, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666281723002020

[21] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24824–24837. [Online]. Available: https://arxiv.org/abs/2201.11903

[22] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[23] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, art. 248, pp. 1–38, Mar. 2023, doi: 10.1145/3571730. [Online]. Available: https://dl.acm.org/doi/10.1145/3571730

[24] M. Scanlon, F. Breitinger, C. Hargreaves, J.-N. Hilgert, and J. Sheppard, "ChatGPT for digital forensic investigation: The good, the bad, and the unknown," *Forensic Science International: Digital Investigation*, vol. 46, art. 301609, Oct. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S266628172300121X

[25] M. Habibzadeh et al., "Large language models for security operations centers: A comprehensive survey," *Journal of Electrical and Computer Engineering*, 2026, doi: 10.1155/jece/3383674. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674

[26] "Possibilities and limitations of using large language models (LLMs) for alert classification and prioritisation in security operations centers (SOCs)," *Expert Systems with Applications*, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417426021032

[27] "LLMs in the SOC: An empirical study of human–AI collaboration in security operations centres," arXiv preprint arXiv:2508.18947, 2025. [Online]. Available: https://arxiv.org/abs/2508.18947

[28] Open Worldwide Application Security Project, *OWASP Top 10 for Large Language Model Applications*, OWASP Foundation. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[29] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61. [Online]. Available: https://pandas.pydata.org/

[30] S. Ramírez, "FastAPI — modern, fast web framework for building APIs with Python." [Online]. Available: https://fastapi.tiangolo.com/

[31] ReportLab, "ReportLab PDF toolkit." [Online]. Available: https://www.reportlab.com/opensource/

Appendix A

This shows how you can obtain the initial acquisition — this is where the Amcache, Prefetch, and ShimCache evidence used later in this report was collected. Our approach was to spin up a virtual machine, download malware samples from MalwareBazaar, and run several Metasploit-generated (msfvenom) payloads on the machine. After execution, we captured a full image of the VM using FTK Imager. That image was then transferred to a separate analysis machine, where the relevant artifacts (Amcache, ShimCache, Prefetch, etc.) were extracted for review.

Tools Used

Tool	**Purpose**	**Version Used**
FTK Imager	Physical drive imaging and manual artifact extraction	4.7.1
AmcacheParser	Parses Amcache.hve for execution/existence evidence	2026.5.0
PECmd	Parses Windows Prefetch (.pf) files	Latest EZ Tools
AppCompatCacheParser	Parses ShimCache (AppCompatCache) from SYSTEM hive	Latest EZ Tool s

Table 13 - Tools Utilized to gain initial acquisition

Artifact	Default Location
Amcache	C:\Windows\AppCompat\Programs\Amcache.hve
ShimCache (AppCompatCache)	Registry value in C:\Windows\System32\config\SYSTEM, key: ControlSet00X\Control\Session Manager\AppCompatCache
Prefetch	C:\Windows\Prefetch*.pf

Table 14 - Artifact Locations Referenced

Malware Execution / Evidence Gathering

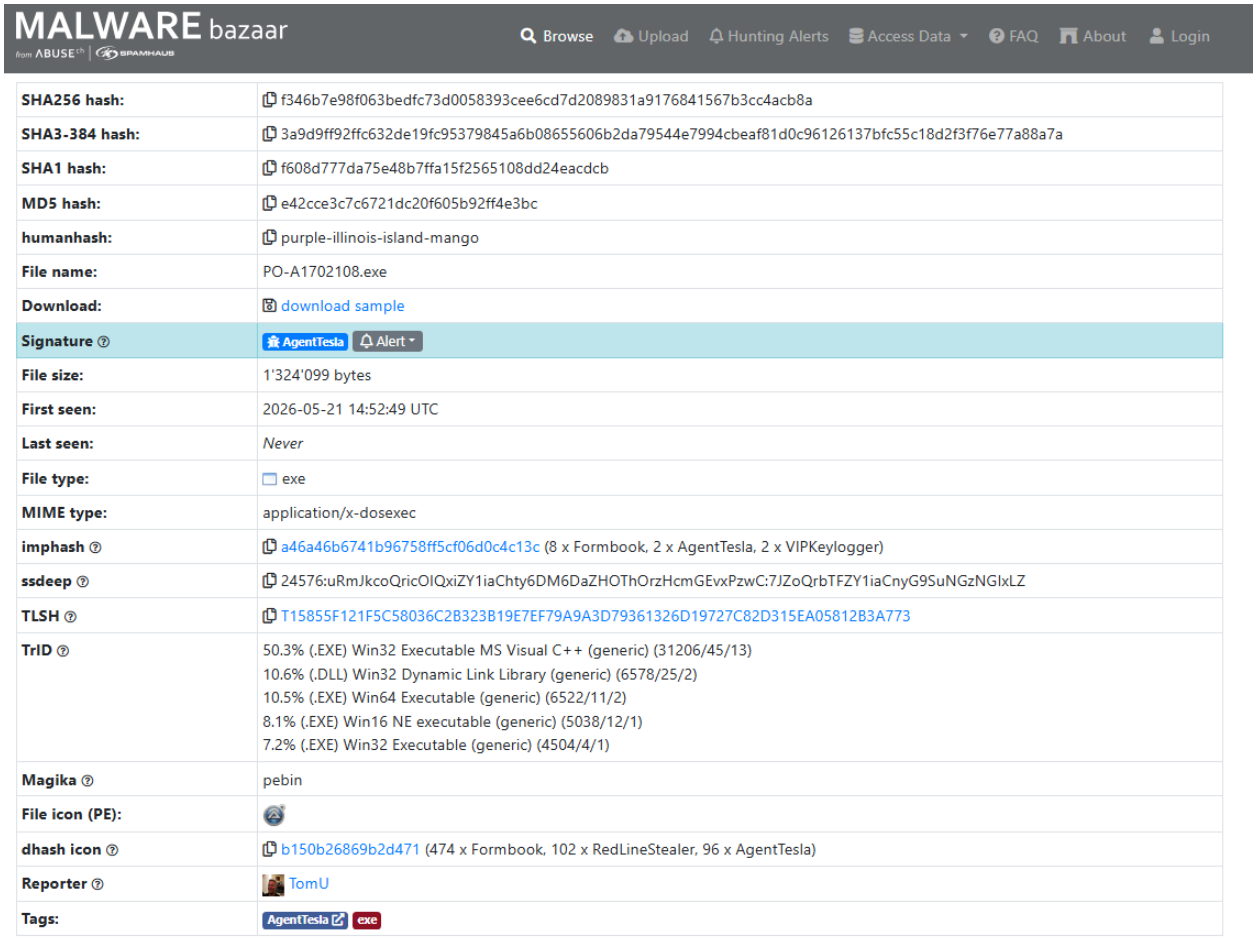


Figure 11 - Malware Executed on the Isolated Victim Machine

After disconnecting the network adapter and ensuring the VM was isolated, we extracted the malware from the password-protected zip file and ran it on the victim machine.

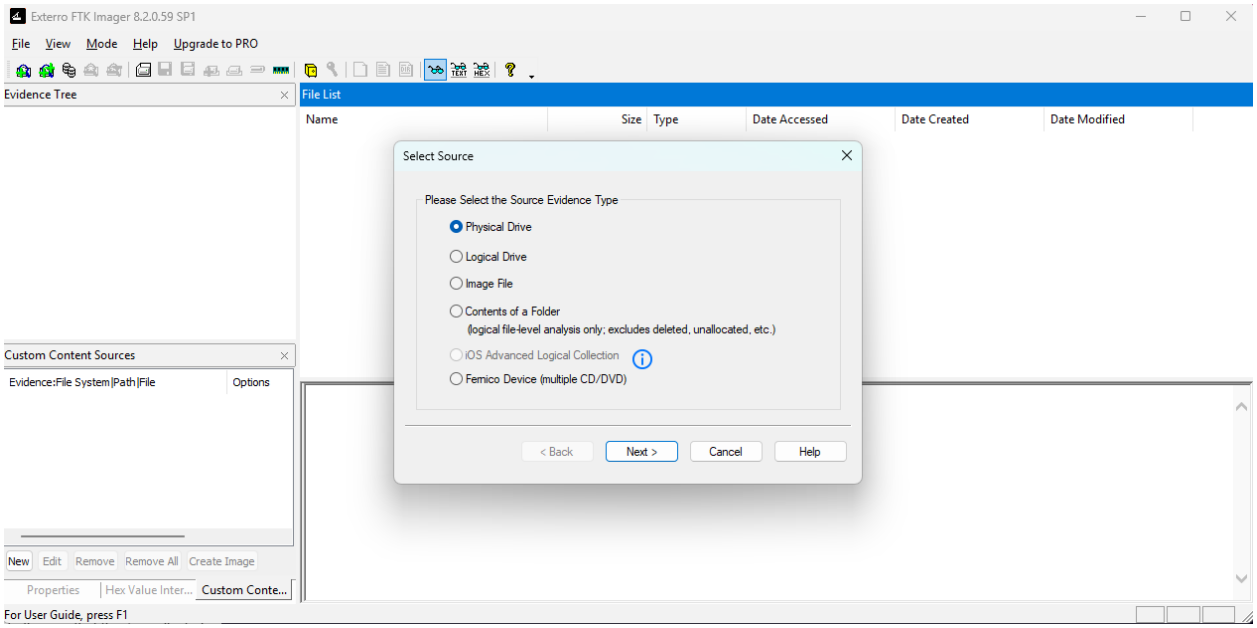


Figure 12 - Creating a Physical Drive Image in FTK Imager

This screenshot shows that we will be creating a physical drive image with FTK Imager; this will provide us with physical evidence of the actual Windows 11 VM.

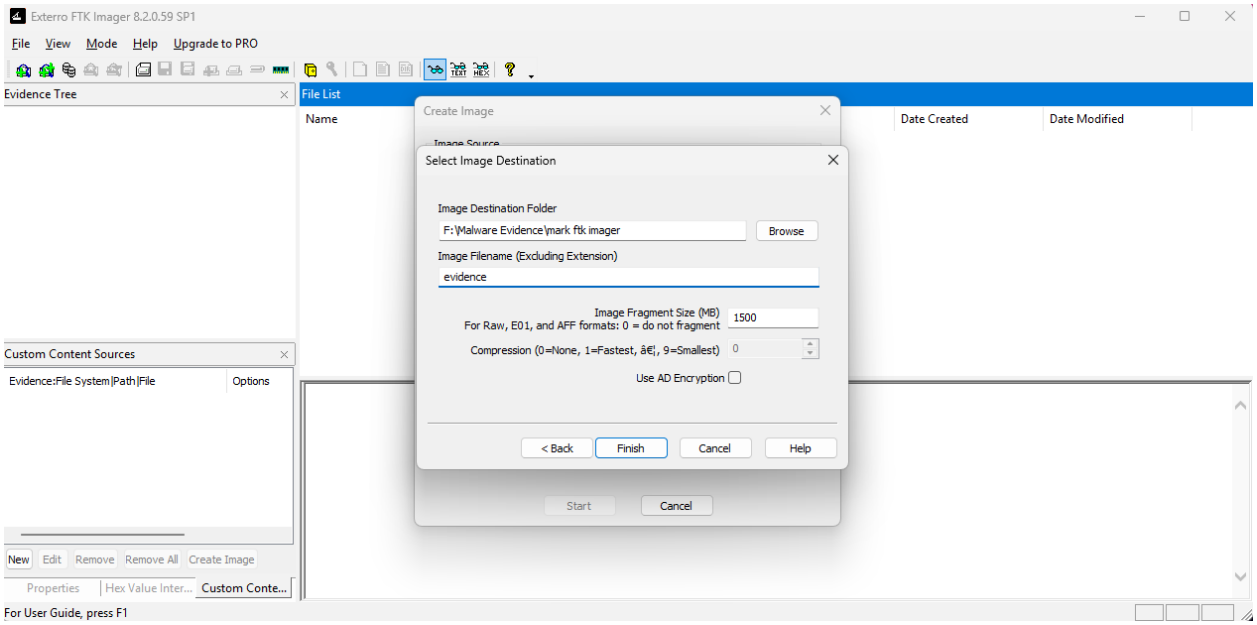


Figure 13 - FTK Imager Destination Folder and Image File Name

The figure above shows a configuration in which the path for the destination folder is specified as F:\Malware Evidence\mark ftk imager, and the image file name is specified as “evidence.” This is the location where the forensics image will be saved.

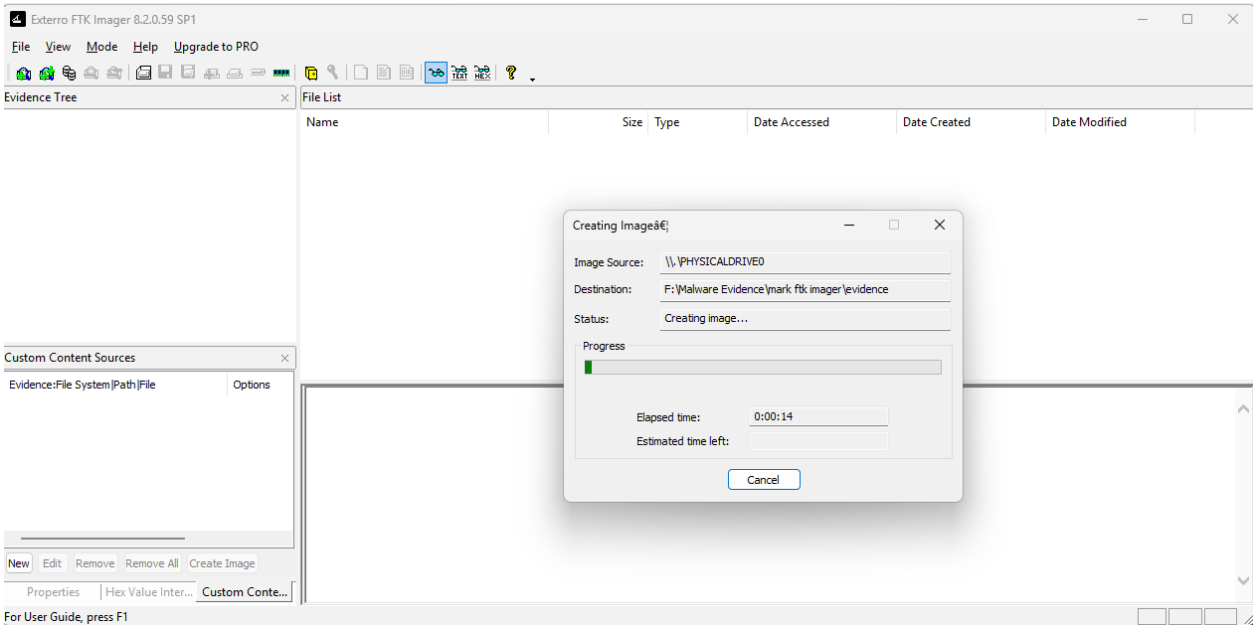


Figure 14 - FTK Imager Imaging in Progress

This screenshot shows the imaging process being done. Using the tool FTK Imager, we were able to create the image from the physical drive source (\\.\PHYSICALDRIVE0) to the destination folder.

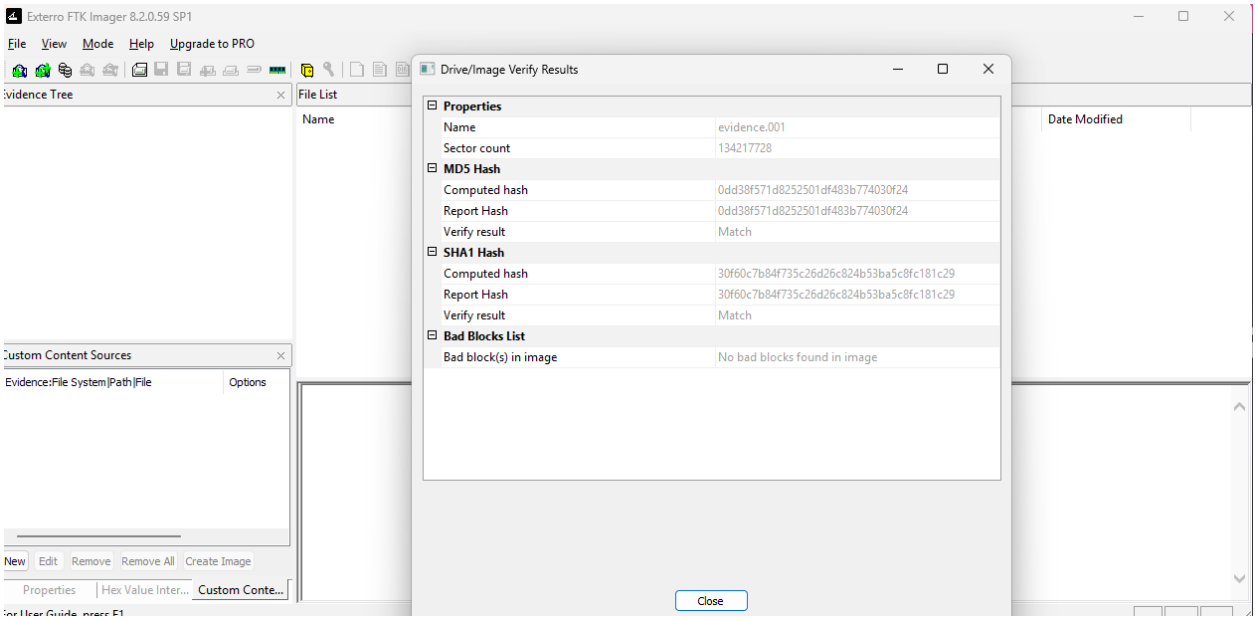


Figure 15 - Drive/Image Verify Results

The above image shows the "Drive/Image Verify Results," which indicates that the MD5 & SHA1 hash results generated after imaging match the report's hash results, and no bad sectors were detected in the image file. This was provided after the imaging was completed.

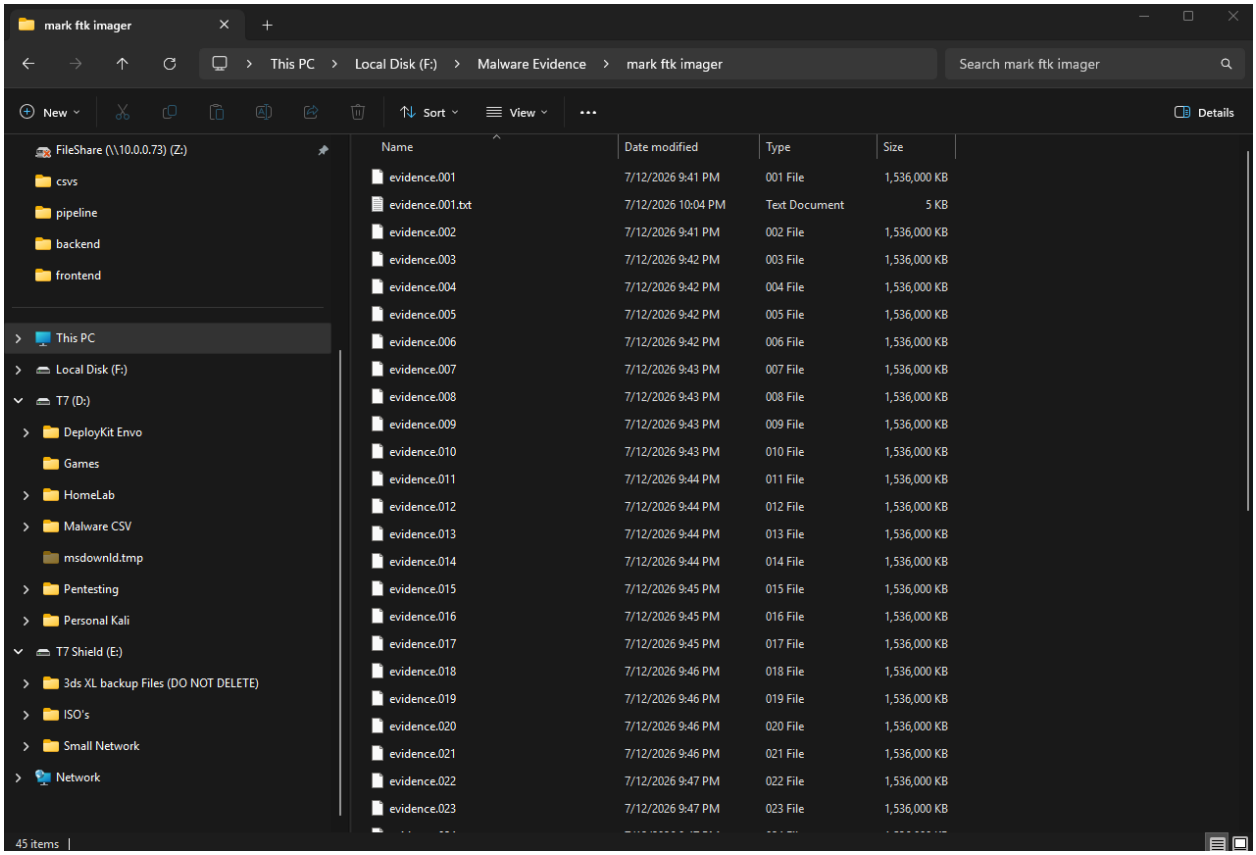


Figure 16 - Image Segments in Windows Explorer

This screenshot shows the destination folder in Windows Explorer, where the image was split into multiple segments.

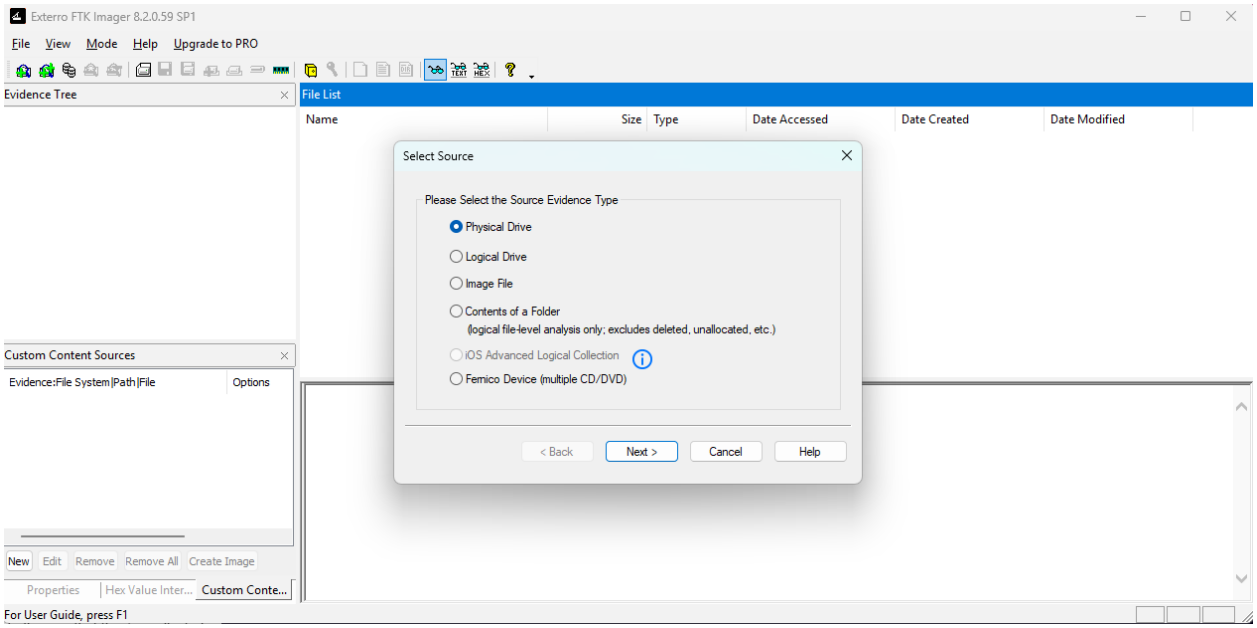


Figure 17 - Loading the Evidence Image as a Source in FTK Imager

This screenshot shows the "Select Source" dialogue in FTK Imager, where we will be loading the previously created evidence image (evidence.001) back into FTK Imager. This will allow us to grab the Amcache, Shimcache, Prefetch files and other evidence that may be needed for our forensics tool.

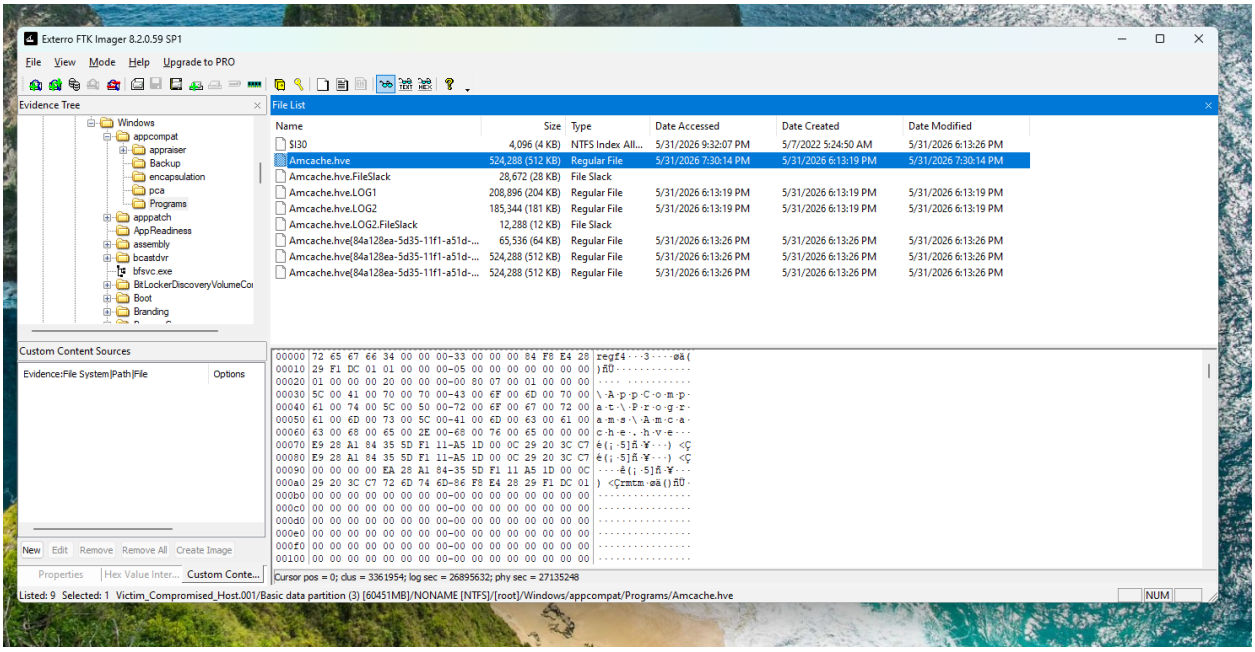


Figure 18 - Extracting Windows Artifacts from the Evidence Image

This shows that after we obtained all the raw image segments (totalling the size of the Hard Drive), we manually extracted the logs from the images using FTK Imager. This helped us obtain a variety of Windows artifacts that require investigation. For example, here you can see how the Amcache.hve was found and later exported. This was applied to other similar artifacts.